STICK ARENA

Frontend and Backend Architecture

A detailed visual guide to the Godot 4.7 local fighting-game prototype

FRONT END | PLAYER-FACING SYSTEMS

Screens, menus, keyboard navigation, HUD, scenes, arenas, character presentation, camera, particles, and sound.

BACK END | LOCAL GAME LOGIC

Input interpretation, movement, combat, CPU decisions, weapons, super moves, timers, rounds, Practice Mode, and hazards.

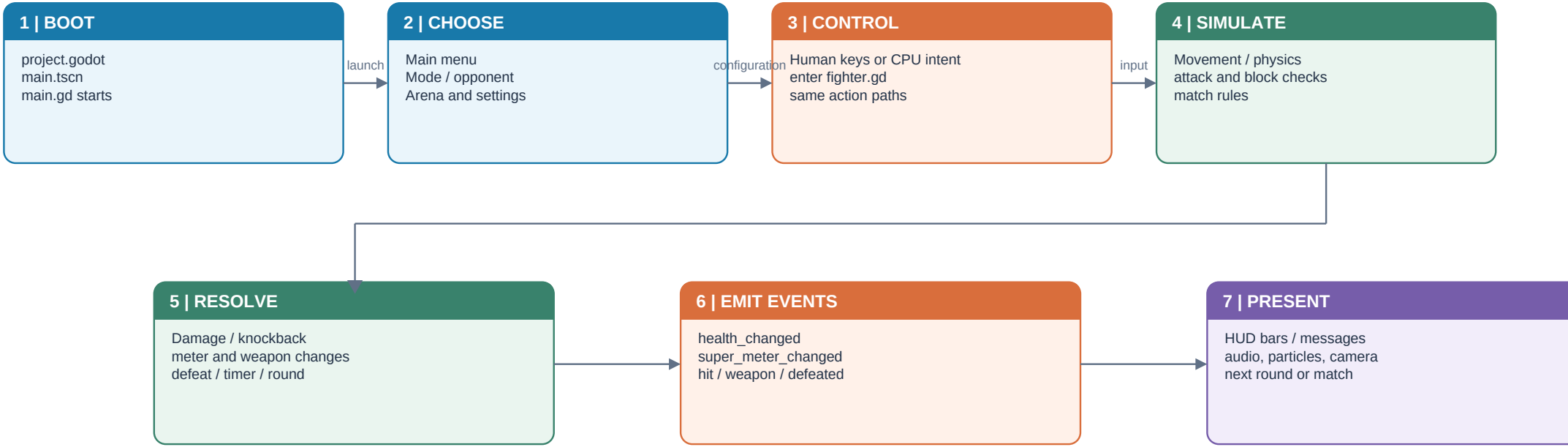
60 pages | one system or flow per diagram page

Important: this project has no online multiplayer server. In this guide, “backend” means the local game rules and logic running inside Godot.

How the whole project fits together

Follow the arrows from launch to presentation. Front-end objects show the interface and visuals; back-end objects decide what the game does.

Interface / scenes · Controller / rules · Fighter / physics · Effects / feedback



READ THIS MAP

The main controller coordinates the match, but each Fighter node owns its immediate physics and combat response. The HUD listens to changed values; it does not calculate attack outcomes.

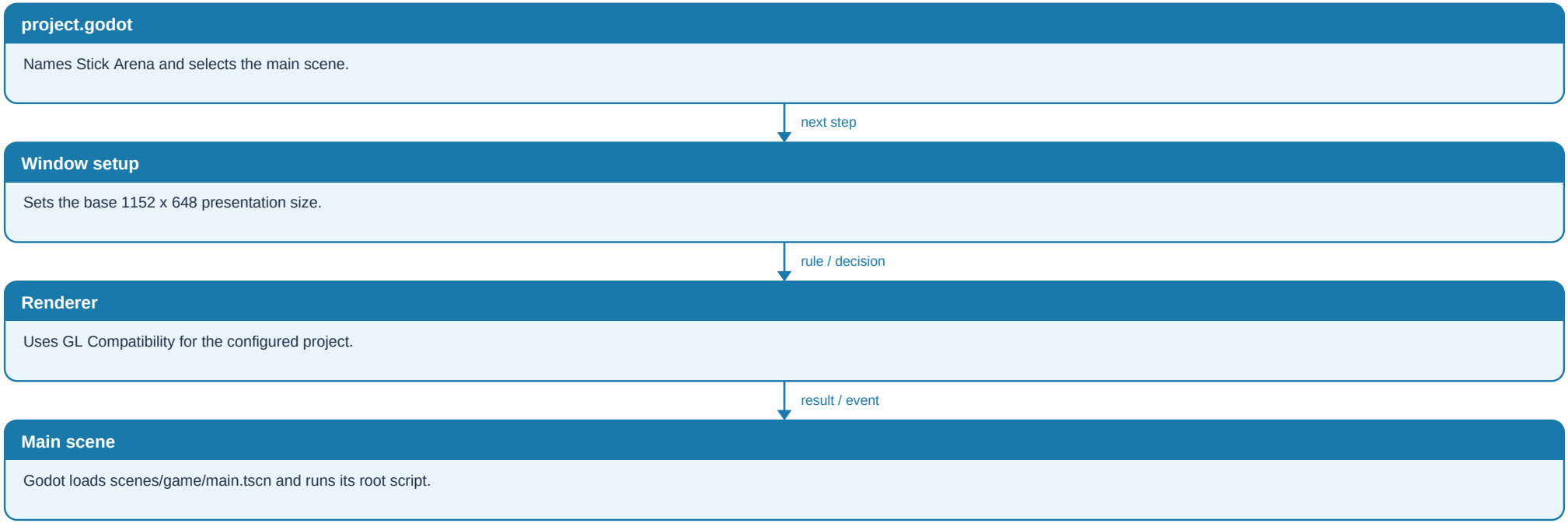
HOW THE DETAILED PAGES ARE ORGANIZED

Pages 3-27 describe the player-facing side. Pages 28-55 describe local game logic and integration. Each page has a four-step flow, a short explanation, and the project files to inspect.

1. Project boot and renderer configuration

The game begins as a Godot project before any match screen is shown.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

This page distinguishes project-level startup from game-level navigation. Startup configuration controls which scene is loaded and how the window is presented; it does not select a game mode or start a fight.

HOW THE FLOW WORKS

The `run/main_scene` setting points to the main scene. From there, the root node's `_ready()` setup creates initial screen visibility, HUD values, audio objects, and the main menu state.

PRESENTER NOTE

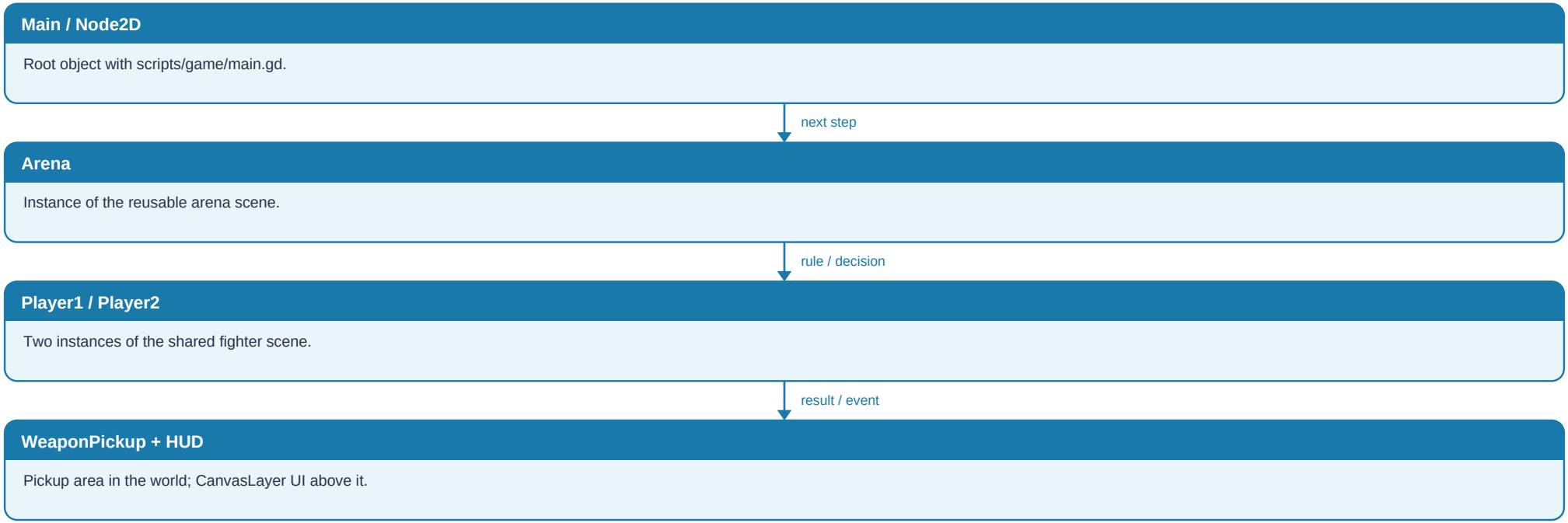
Point out that the main scene is the entry point. The menu is shown after launch; a match begins only after the player makes selections.

PROJECT FILES	project.godot | scenes/game/main.tscn | scripts/game/main.gd

2. Main scene composition

The main scene assembles the match world and screen-space interface.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The scene file establishes the object tree before runtime behavior starts. Reusing the same fighter scene keeps the two players structurally consistent while exported properties provide player number and color.

HOW THE FLOW WORKS

At runtime, main.gd links the two fighters as opponents, connects their signals, and coordinates the arena and HUD nodes. The HUD is a CanvasLayer, so it stays aligned with the screen while world objects move.

PRESENTER NOTE

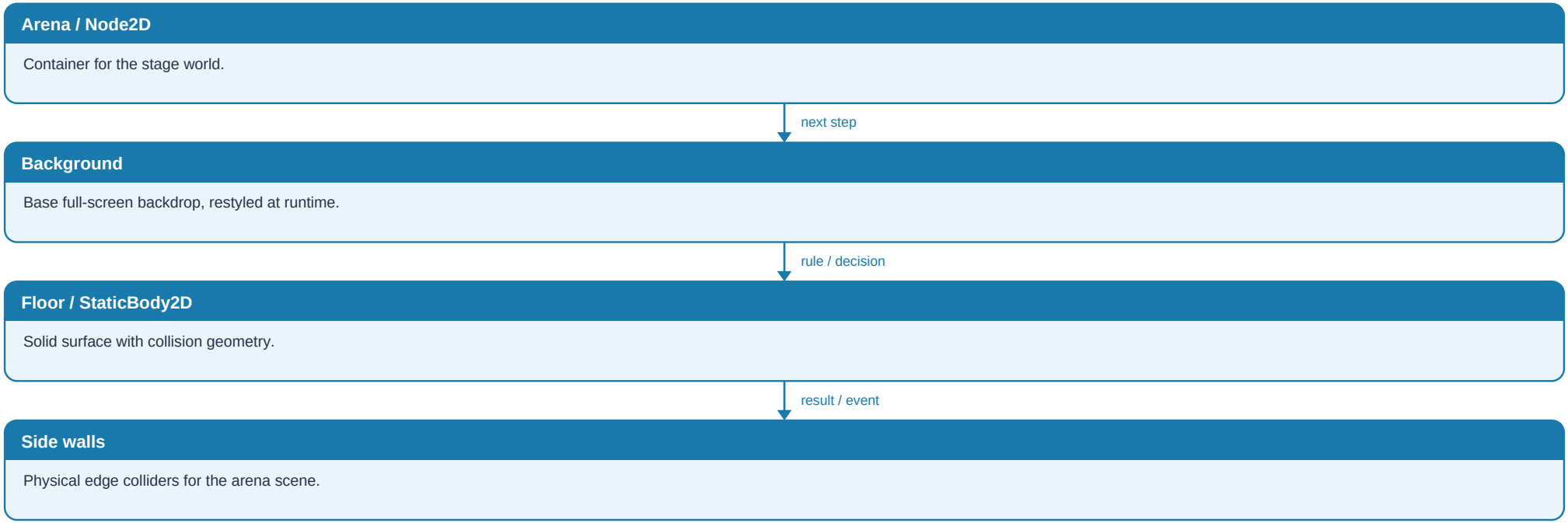
Show the scene tree and explain which nodes belong to the world and which belong to the interface.

PROJECT FILES scenes/game/main.tscn | scenes/player/fighter.tscn | scenes/arenas/arena.tscn

3. Arena scene and world layer

The arena scene supplies collision and reusable world elements.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The arena scene gives the game a stable base world. The controller can change visual styling and, for Arena 2, adjust the floor/hazard presentation and fighter behavior without replacing the entire game scene.

HOW THE FLOW WORKS

Visual geometry and physical collision serve different purposes: floor and wall shapes constrain bodies, while Polygon2D, Line2D, and generated shaders provide the look.

PRESENTER NOTE

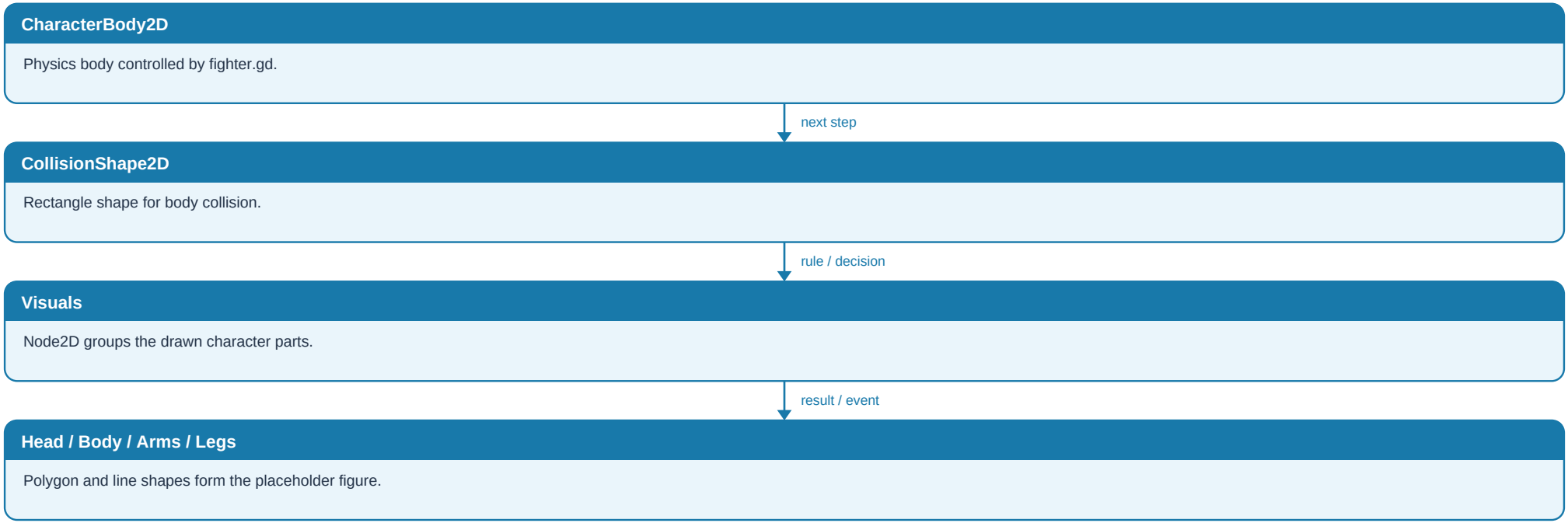
Emphasize that Arena 2 is a selectable variation managed by the game rather than a different combat engine.

PROJECT FILES scenes/arenas/arena.tscn | scripts/game/main.gd

4. Fighter scene and reusable character

One fighter scene is used for both human players and CPU/dummy roles.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The scene separates collision from appearance. The CharacterBody2D can move and collide while the Visuals child changes color, scale, pose, and rotation independently.

HOW THE FLOW WORKS

Player 1 and Player 2 share this packed scene. The game sets their number and tint in main.tscn; fighter.gd uses player_number to select controls and opponent identity.

PRESENTER NOTE

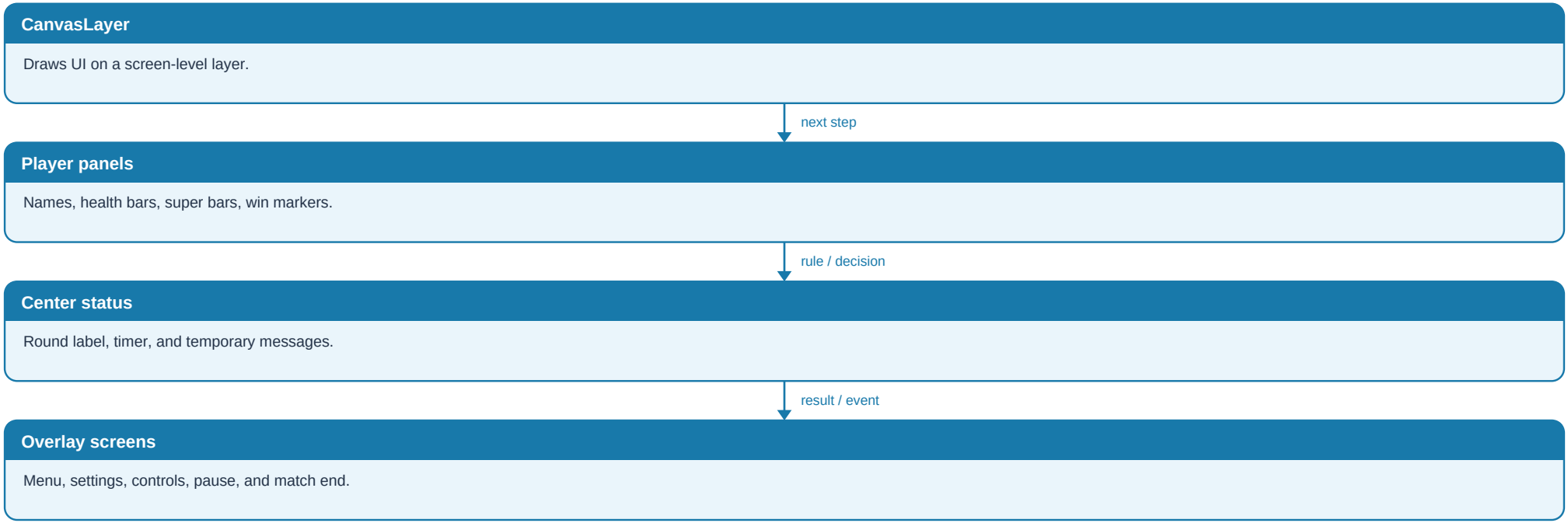
If asked why simple shapes are used, explain they keep the prototype asset-free and make the game easy to modify.

PROJECT FILES scenes/player/fighter.tscn | scripts/player/fighter.gd | scenes/game/main.tscn

5. HUD hierarchy and screen-space layer

The HUD displays match state without moving with the arena camera.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The HUD includes persistent widgets and full-screen Control overlays. main.gd decides which overlays are visible and updates their text or values.

HOW THE FLOW WORKS

Use the CanvasLayer boundary to explain why bars remain fixed while the camera frames both fighters. Overlays can cover the arena view without deleting the world nodes.

PRESENTER NOTE

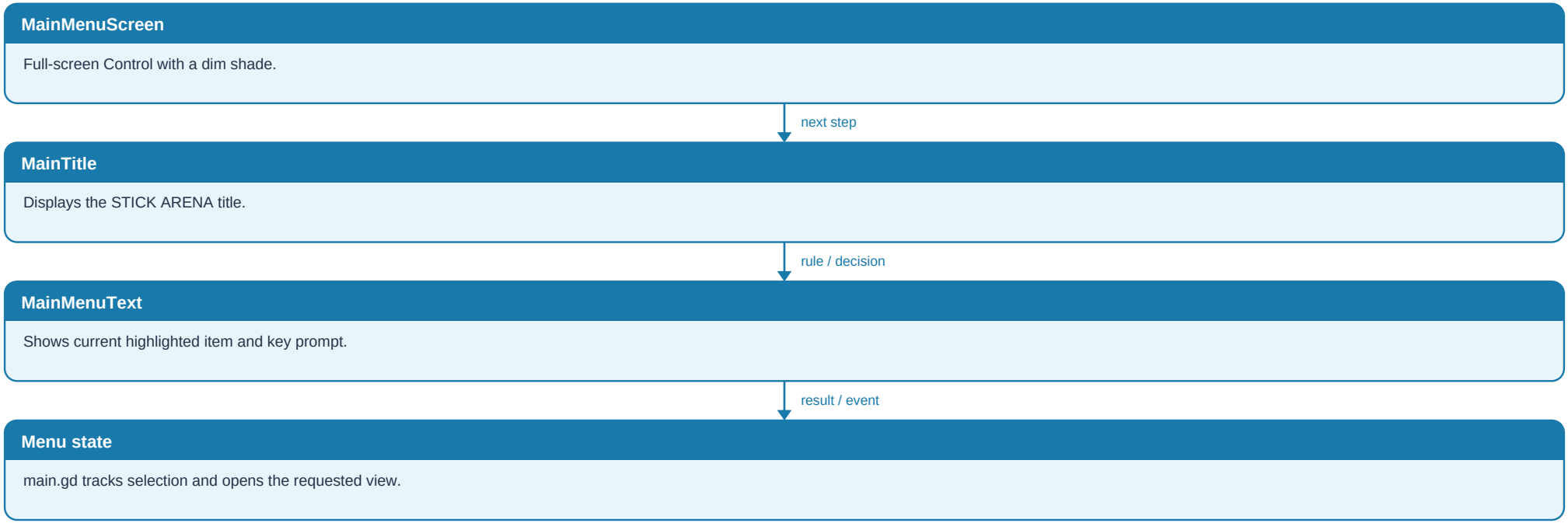
Distinguish displaying a value from owning that value: health belongs to Fighter, while ProgressBar presents it.

PROJECT FILES scenes/game/main.tscn | scripts/game/main.gd

6. Main menu screen

The main menu is the launch point for Play, Settings, and Controls.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The menu is a visual representation of the current menu state. A selection change refreshes the text/highlight; Enter activates the selected action.

HOW THE FLOW WORKS

Play moves to mode selection. Settings and Controls show their own screens. Escape is intentionally inert at the top level.

PRESENTER NOTE

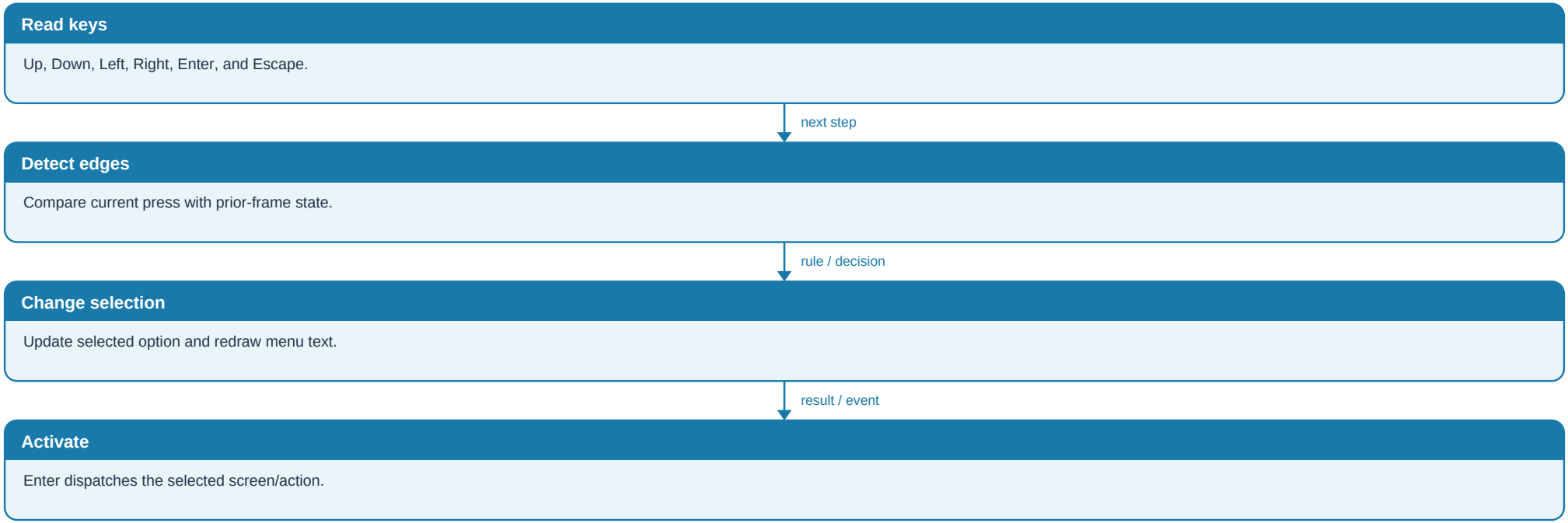
Demo: start the project and point out that it opens on the title menu, not inside a fight.

PROJECT FILES scenes/game/main.tscn | scripts/game/main.gd

7. Keyboard menu navigation

Menus use edge-detected key presses so a held key does not repeatedly activate every frame.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The controller stores prior pressed states such as up_was_pressed and enter_was_pressed. Comparing states creates a one-shot just-pressed event from the raw keyboard poll.

HOW THE FLOW WORKS

Navigation uses discrete actions for menu items. Left/Right are used where the screen exposes a horizontal choice or setting value.

PRESENTER NOTE

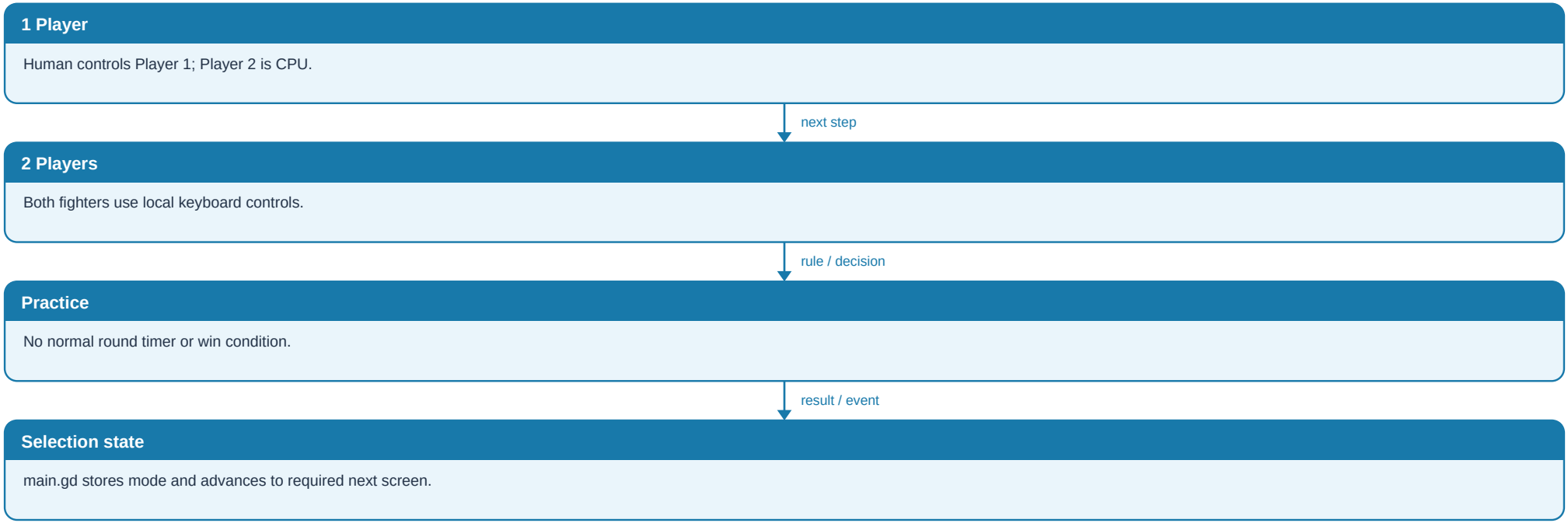
This pattern prevents accidental repeat navigation when a key is held down.

PROJECT FILES scripts/game/main.gd

8. Mode-selection screen

The mode screen chooses the type of session before stage selection.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Mode selection determines how the two existing fighter nodes are configured. The core scenes remain the same; controller flags change who provides input and how match rules run.

HOW THE FLOW WORKS

Practice continues to opponent choice before stage selection. Normal modes proceed through CPU difficulty or the available mode options, then stage selection.

PRESENTER NOTE

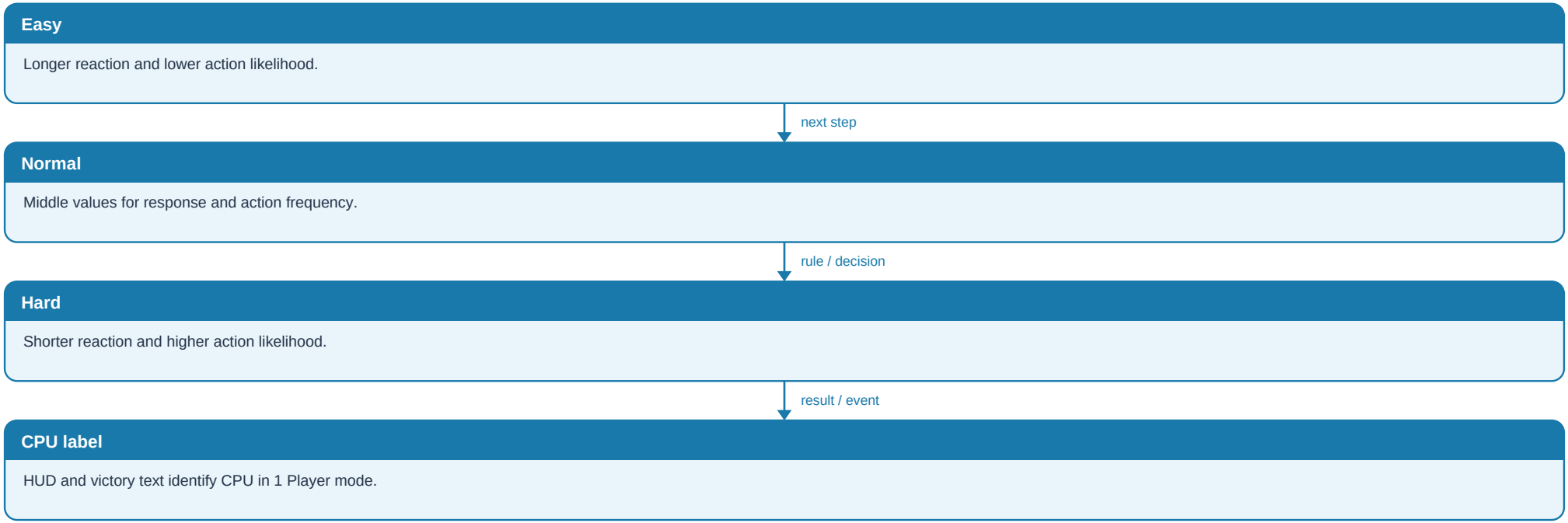
Describe modes as configuration choices rather than separate games.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

9. CPU difficulty selection

Difficulty is a player-facing option that maps to CPU behavior parameters.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The screen communicates a choice; main.gd stores the selected difficulty and uses indexed parameter arrays for reaction, blocking, attack, and special behavior.

HOW THE FLOW WORKS

Difficulty is intended to affect CPU decisions, not the human control map or the base normal attack damage values.

PRESENTER NOTE

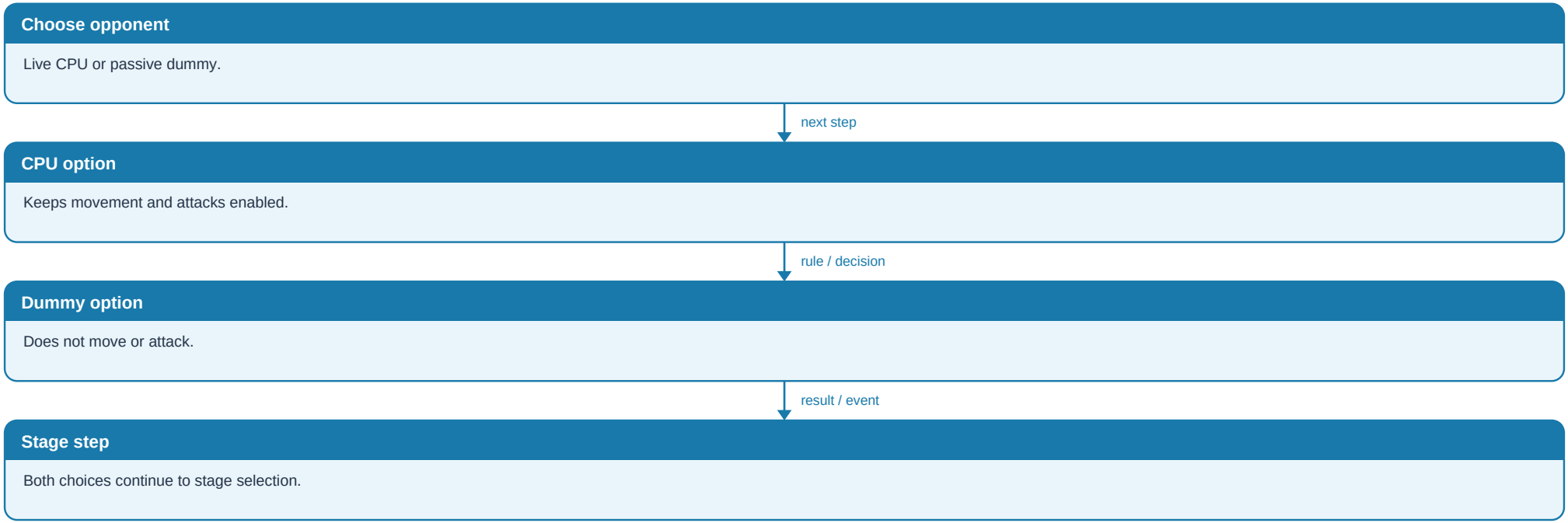
Keep the explanation focused on fairness settings rather than claiming the CPU learns or uses machine learning.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

10. Practice opponent choice

Practice can use a live CPU or a passive dummy.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The dummy is still a Fighter node, so ordinary hits, blocking rules, hit reactions, and collision can be demonstrated without writing a different damage target.

HOW THE FLOW WORKS

The CPU option uses the chosen difficulty. The dummy option turns off its behavior while leaving it available for hits and practice.

PRESENTER NOTE

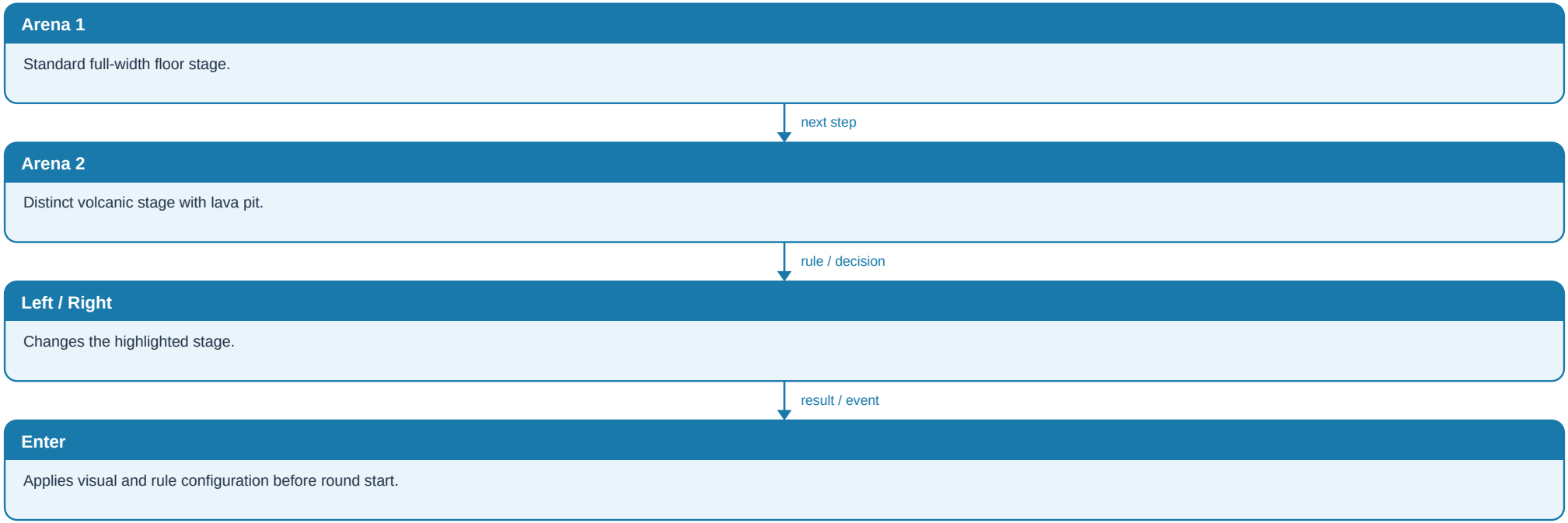
Explain why a dummy is useful: it lets a player test combat responses without an opponent interrupting.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

11. Stage-selection screen

Stage selection applies to both normal modes and Practice Mode.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The selected stage is stored centrally and applied to arena visuals, floor collision settings, fighter hazard flags, and weapon behavior.

HOW THE FLOW WORKS

The stage is not just a background choice: Arena 2 enables a pit rule; Arena 1 remains the standard full-floor rule set.

PRESENTER NOTE

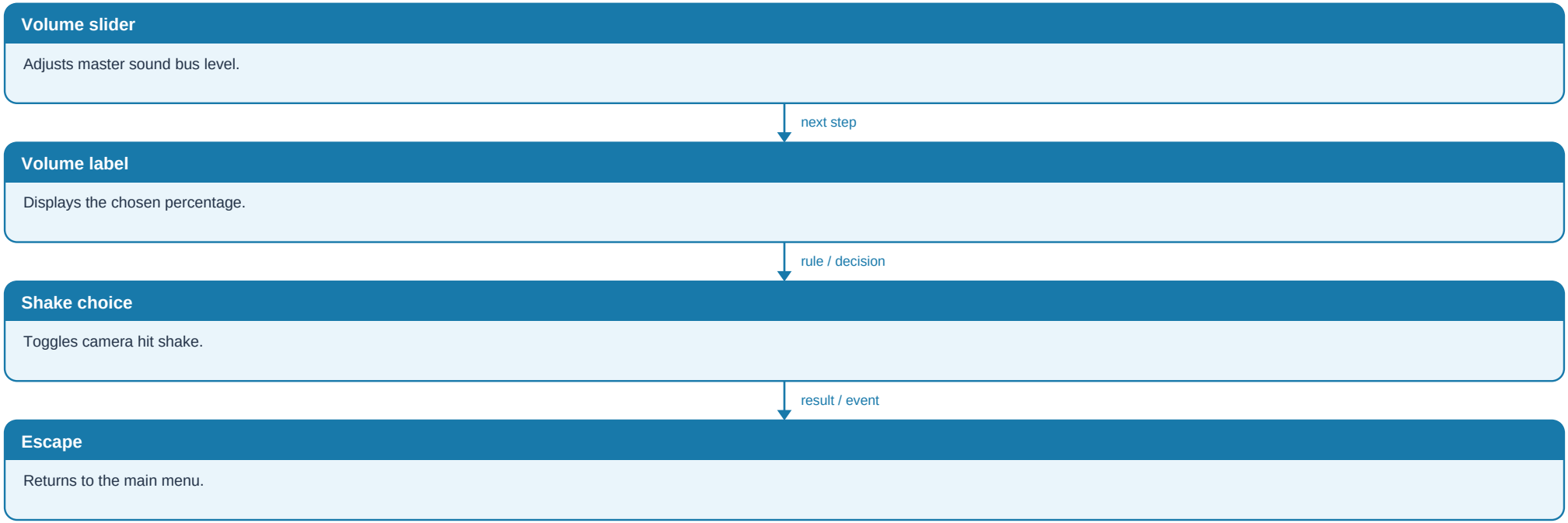
Mention the same bat, fighter controls, and core combat are reused across both arenas.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd | scenes/arenas/arena.tscn

12. Settings screen

Settings expose master volume and hit screen-shake preferences.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The screen is a control surface for two runtime preferences. The volume value is converted to decibels for Godot's audio bus; shake state gates the camera offset strength.

HOW THE FLOW WORKS

Settings do not change damage, speed, cooldowns, or round rules. Their effects are limited to sound output and a presentation option.

PRESENTER NOTE

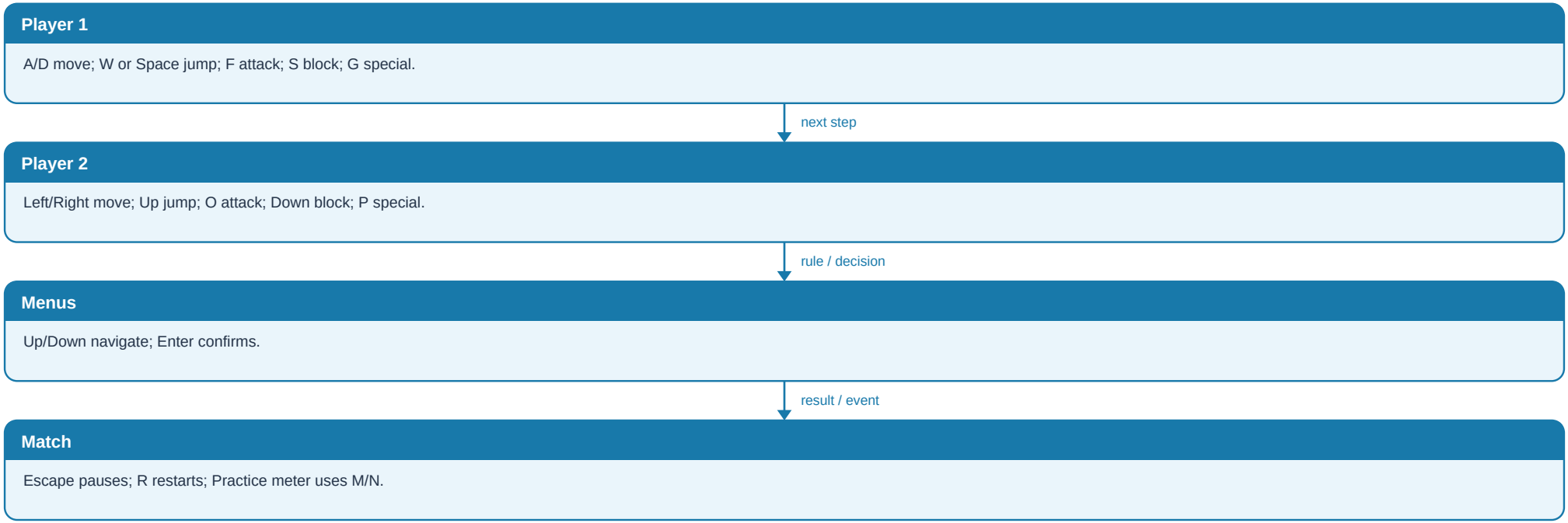
Explain the Left/Right interaction for the focused setting and Escape for returning.

PROJECT FILES	scripts/game/main.gd | scenes/game/main.tscn

13. Controls reference screen

The controls screen provides a static reference for player and match keys.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The display text is a quick reference, while actual input reading is implemented in the scripts. The two uses should stay consistent when controls are changed.

HOW THE FLOW WORKS

The controls page is static and does not remap inputs. The dedicated block actions are configured in the input map; other actions are read from keys.

PRESENTER NOTE

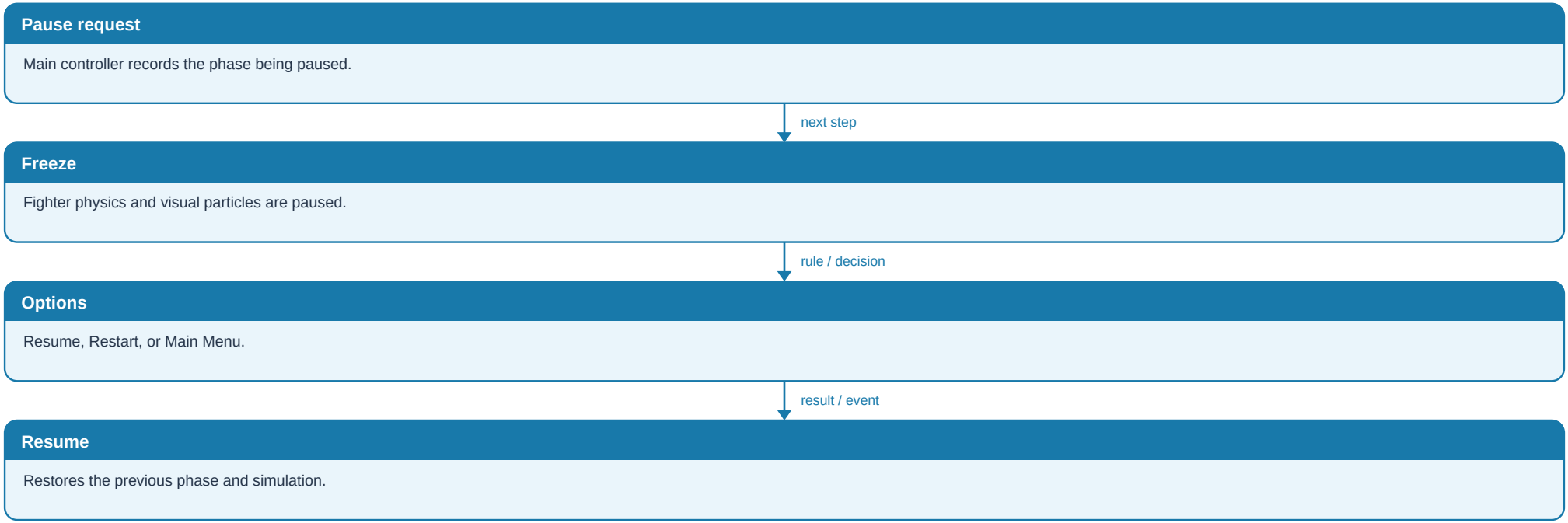
Use this page to orient the teacher before the live match demo.

PROJECT FILES scenes/game/main.tscn | scripts/player/fighter.gd | project.godot

14. Pause overlay

Escape opens a pause menu during an active match or round transition.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Pause is an overlay, not a new scene. The controller saves phase_before_pause and disables fighter physics processing while the menu is active.

HOW THE FLOW WORKS

The round timer is stopped because normal gameplay update is suspended. Resume restores the prior phase; Restart reloads the current scene; Main Menu resets match state.

PRESENTER NOTE

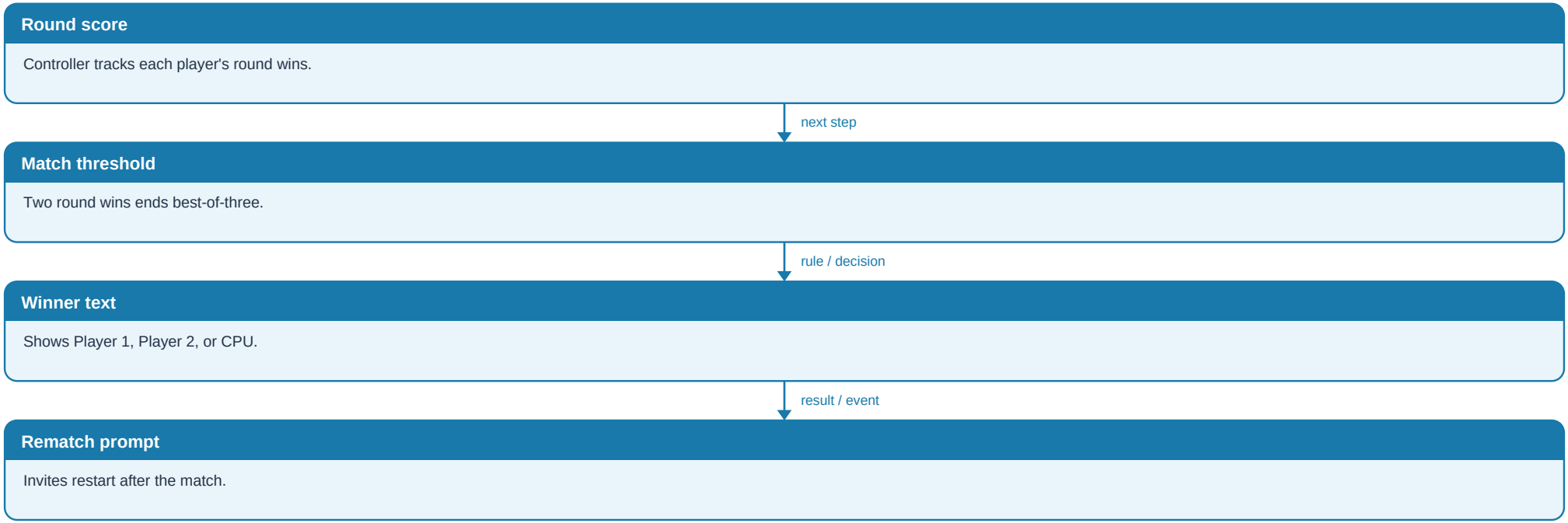
When presenting, show pause and explain that the fight world remains beneath the overlay.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

15. Match-end screen

A match-end overlay announces the winner after one fighter wins two rounds.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The match-end screen covers the arena view and presents the final state. It is driven by the match controller, not by the HUD calculating the winner.

HOW THE FLOW WORKS

The winner label respects single-player mode so the CPU is not presented as Player 2. Practice mode does not use this normal win flow.

PRESENTER NOTE

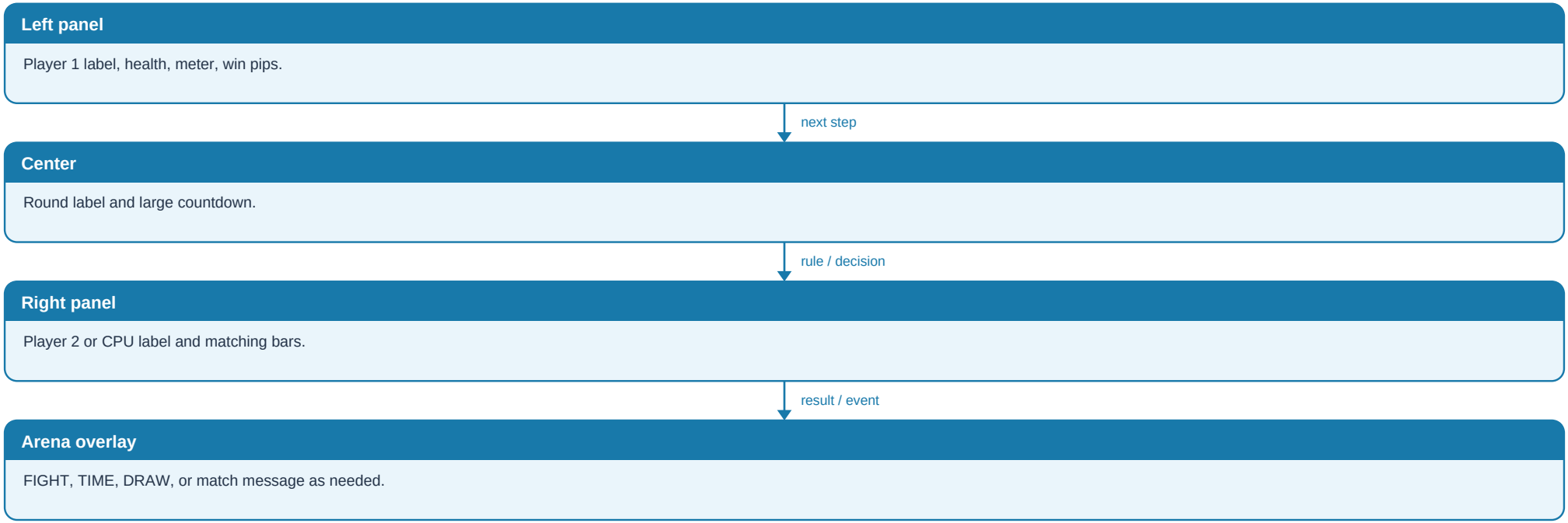
Explain the distinction between a round winner and match winner.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

16. Active-match HUD layout

The HUD puts health, meters, score, time, and temporary messages in stable screen locations.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The panel structure helps players compare state quickly. ProgressBar controls present numerical values while Labels and ColorRects present names, score pips, and momentary status.

HOW THE FLOW WORKS

The center timer is hidden or inactive in Practice Mode. Match messages are temporary and are updated by phase transitions.

PRESENTER NOTE

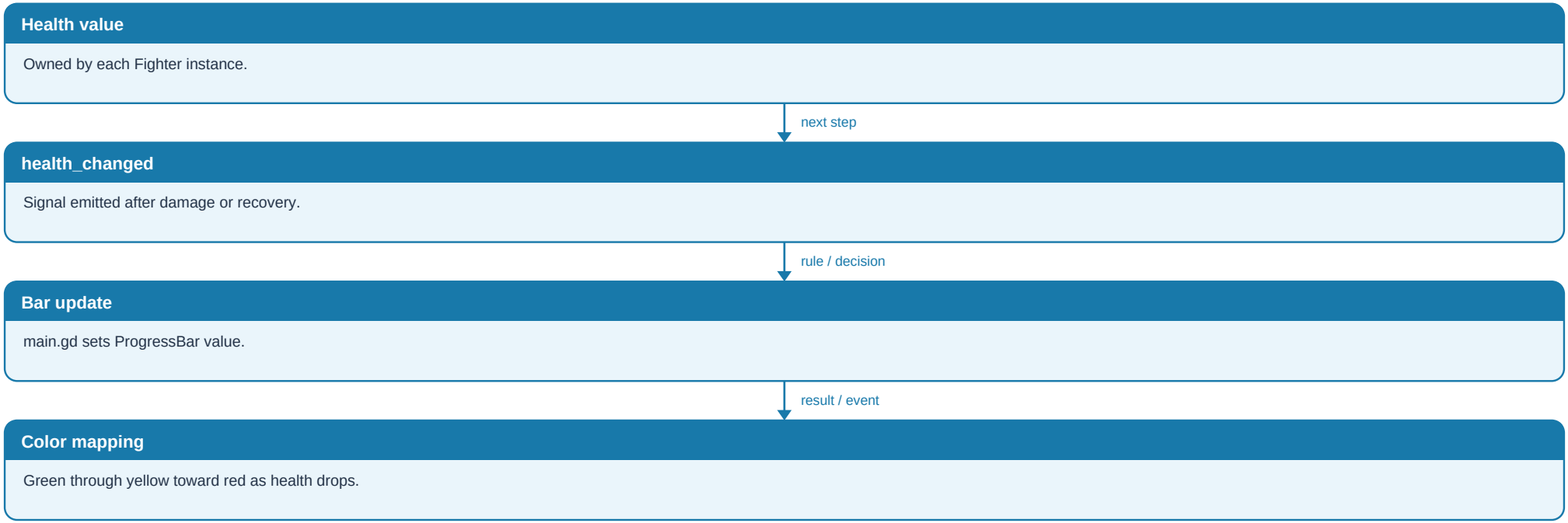
Point from each fighter to its corresponding bar and round-win indicators.

PROJECT FILES scenes/game/main.tscn | scripts/game/main.gd

17. Health-bar presentation

Health is rendered as a shrinking bar whose color changes with remaining health.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The signal path separates health ownership from display. The bar does not apply damage; it reflects the updated Fighter.health value after main.gd receives the signal.

HOW THE FLOW WORKS

At round reset, both fighters return to maximum health and the bars refresh. In Practice Mode, recovery after defeat also refreshes the display.

PRESENTER NOTE

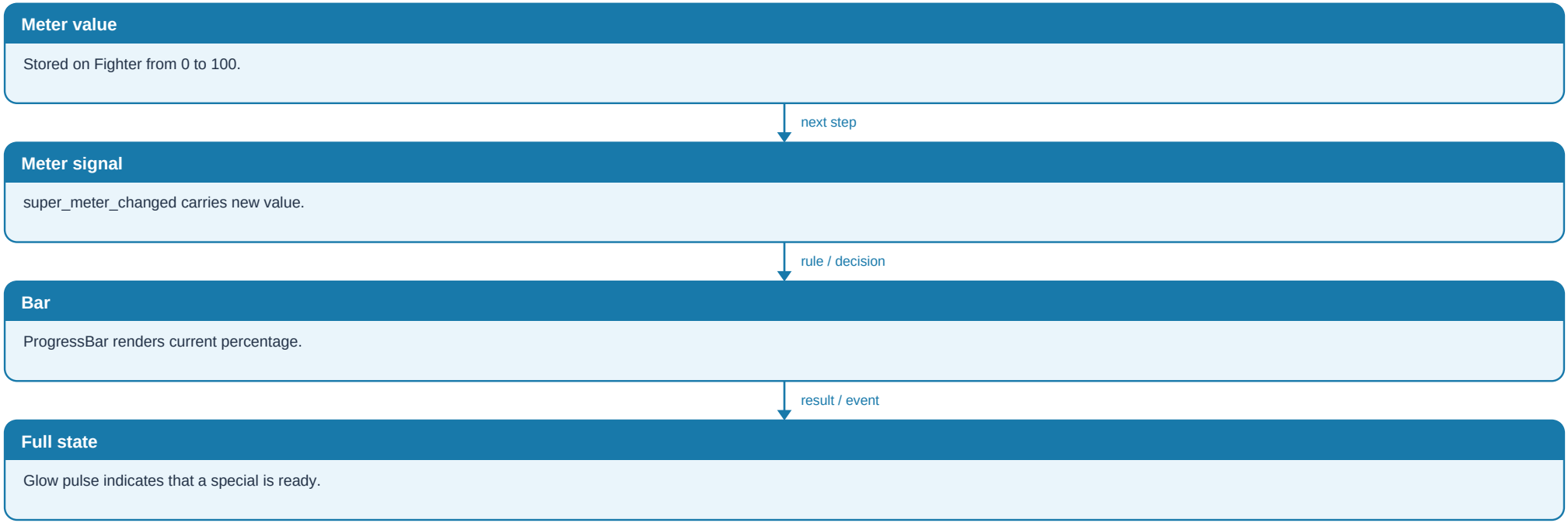
Explain the signal with a simple hit: damage occurs, then the bar changes.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd | scenes/game/main.tscn

18. Super-meter presentation

A small bar under each health bar shows progress toward the special move.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The glow is a visual cue only; special eligibility is decided by Fighter logic when meter is full. The bar helps users understand why a move can or cannot fire.

HOW THE FLOW WORKS

The meter resets at each round. In Practice, a dedicated key can fill a human-controlled fighter's meter to test the special.

PRESENTER NOTE

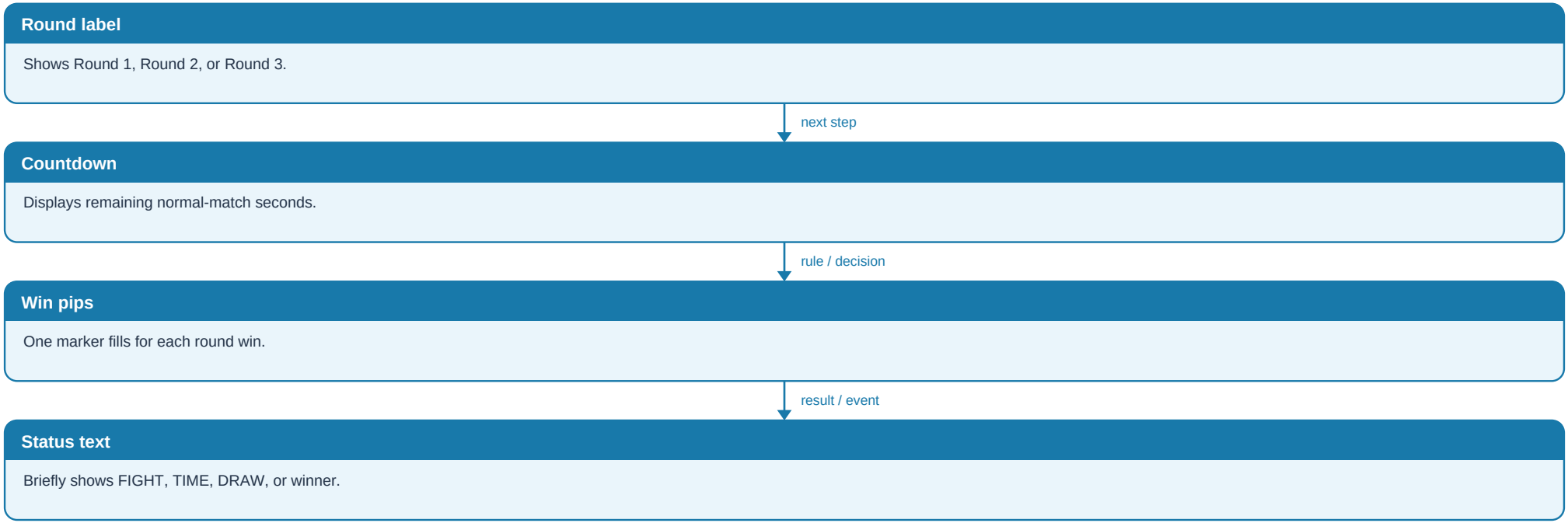
Use M for Player 1 and N for Player 2 in Practice Mode, as shown by the current fighter script.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd | scenes/game/main.tscn

19. Round labels, timer, and win pips

The center display combines the current round, countdown, and score indicators.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The labels are projections of controller state. A draw is handled before incrementing the match score, and the round is replayed.

HOW THE FLOW WORKS

The timer only runs after FIGHT begins and pauses when the game is paused or the round has ended. Practice mode disables the countdown.

PRESENTER NOTE

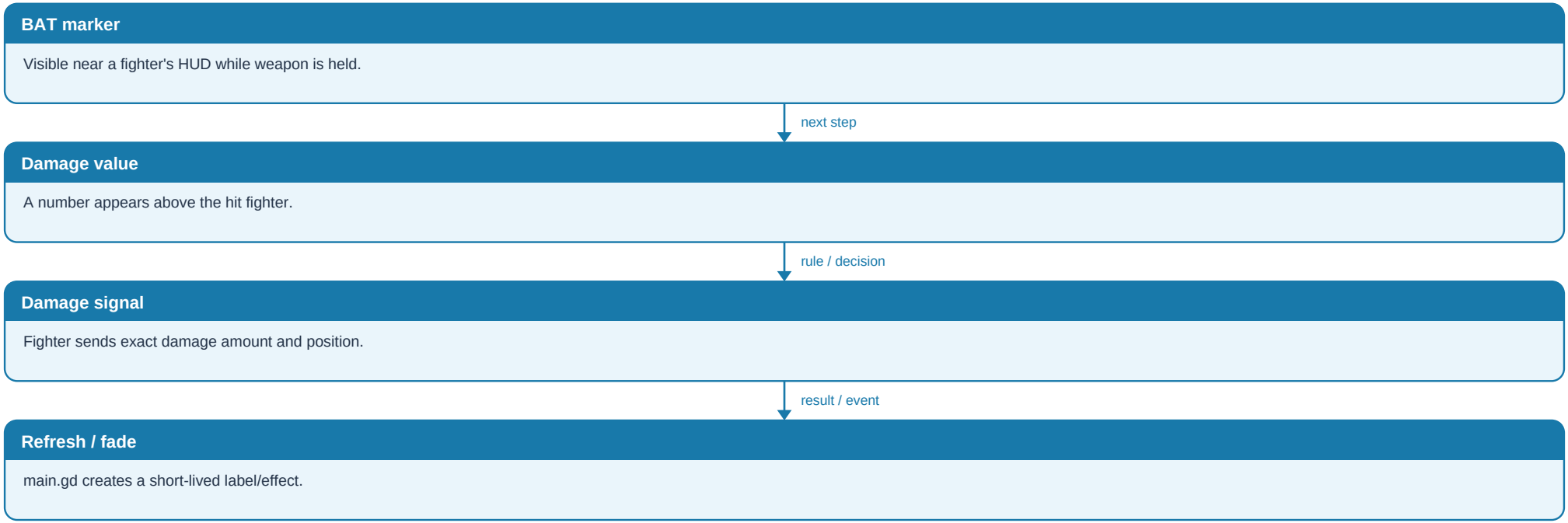
Explain the win pips as round wins rather than health or points for hits.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

20. Weapon indicator and damage number

Small, temporary HUD cues add clarity without changing the combat outcome.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The damage number reports actual health removed, including chip damage and a partially depleted health value. It is informational, not an additional damage system.

HOW THE FLOW WORKS

The bat marker follows weapon-holder state. When hit, the fighter drops the weapon and main.gd updates both indicators.

PRESENTER NOTE

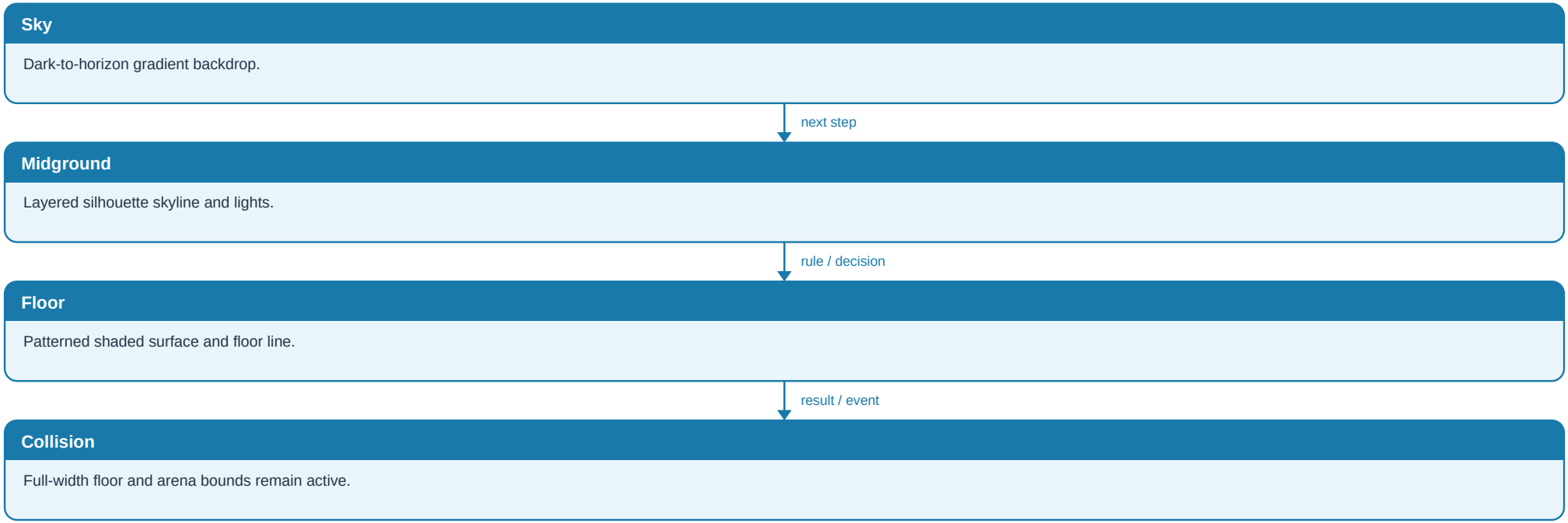
The number helps practice mode communicate what a hit or block actually cost.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd | scenes/game/main.tscn

21. Arena 1 visual composition

Arena 1 is the standard stage with a continuous floor and no hazard damage.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The visuals are generated from built-in Godot nodes and a shader; no external sprite art is required. Arena 1 is the baseline visual and mechanical configuration.

HOW THE FLOW WORKS

The background is built behind the fighters. The floor shader adds pattern/grain while a StaticBody2D provides the actual collision surface.

PRESENTER NOTE

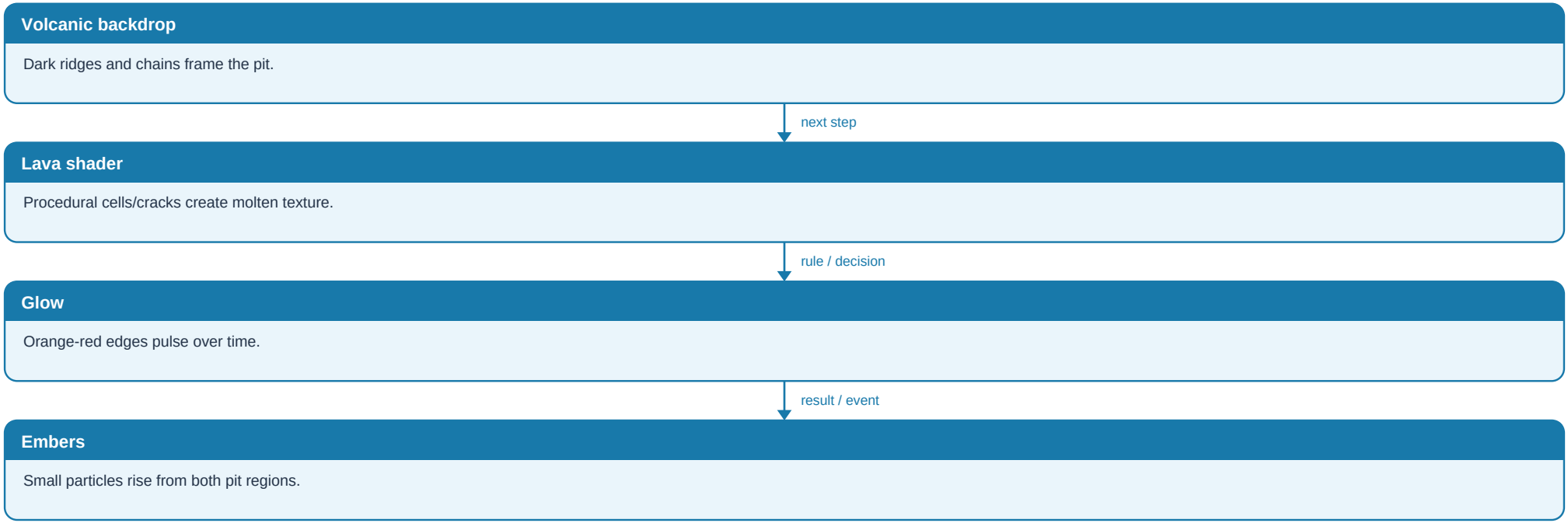
Contrast visual decoration with gameplay collision so the distinction is clear.

PROJECT FILES scripts/game/main.gd | scenes/arenas/arena.tscn

22. Arena 2 lava-stage presentation

Arena 2 changes the stage look and adds a glowing lava pit around a center platform.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Lava is a visual layer as well as a gameplay hazard. The art uses Polygon2D, ShaderMaterial, additive glow, and CPUParticles2D, all built in code.

HOW THE FLOW WORKS

The central safe platform is visually distinct from the pit. A separate gameplay rule checks fighters crossing its edges.

PRESENTER NOTE

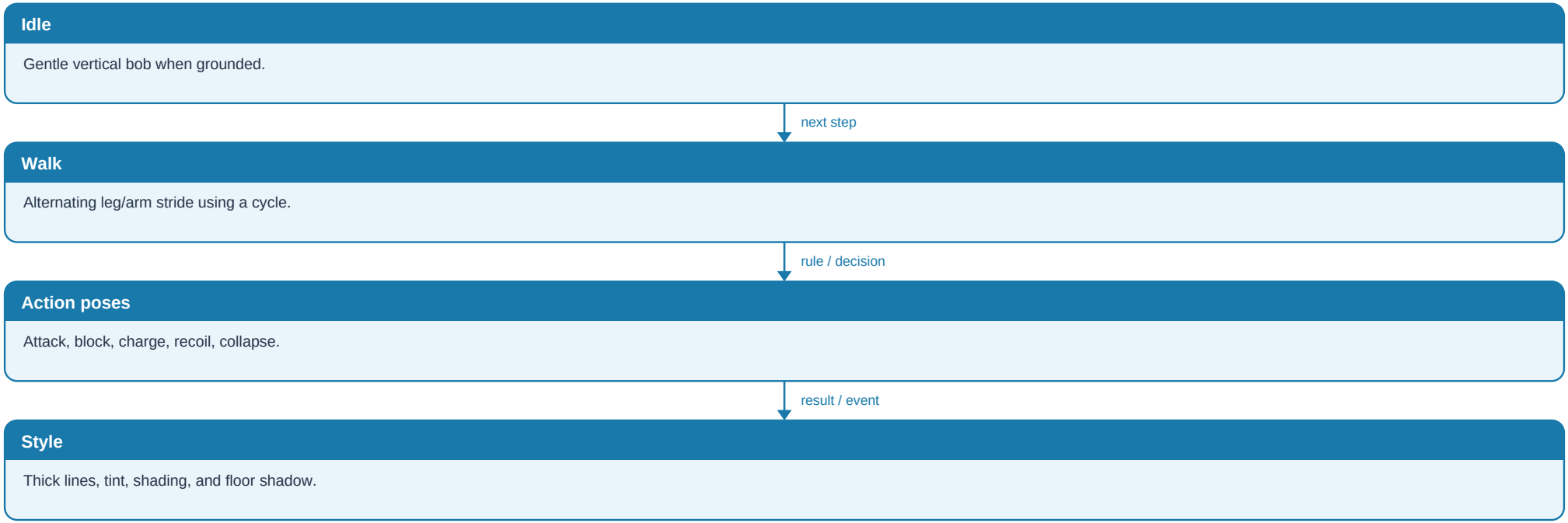
Explain that the shader and particle nodes make this stage self-contained without image assets.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd | scenes/arenas/arena.tscn

23. Fighter poses, shading, and animation

The stick figure is drawn from primitive shapes and changes pose based on fighter state.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Pose updates change only the Visuals child. Movement, health, and hit resolution remain based on the CharacterBody2D and fighter state.

HOW THE FLOW WORKS

Transitions blend line points and rotations for smoother movement. Hit flash temporarily changes color; the normal player tint is restored afterward.

PRESENTER NOTE

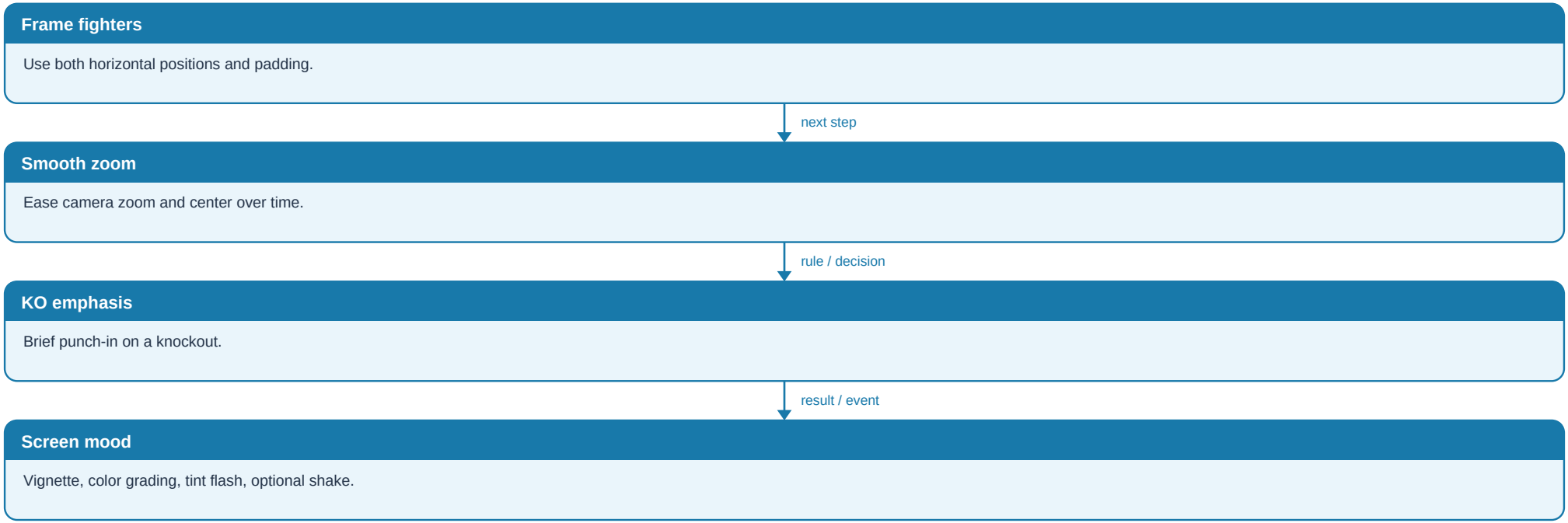
Describe the art as a lightweight procedural character rig rather than a sprite sheet.

PROJECT FILES scripts/player/fighter.gd | scenes/player/fighter.tscn

24. Camera framing and post-processing

The camera keeps both fighters visible and adds short emphasis on impacts.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Camera position and zoom follow the pair of fighters rather than a single character. The pause action resets shake and pauses relevant effects.

HOW THE FLOW WORKS

The screen-shake setting gates hit shake without altering knockback. A CanvasLayer shader applies a subtle vignette and momentary tint.

PRESENTER NOTE

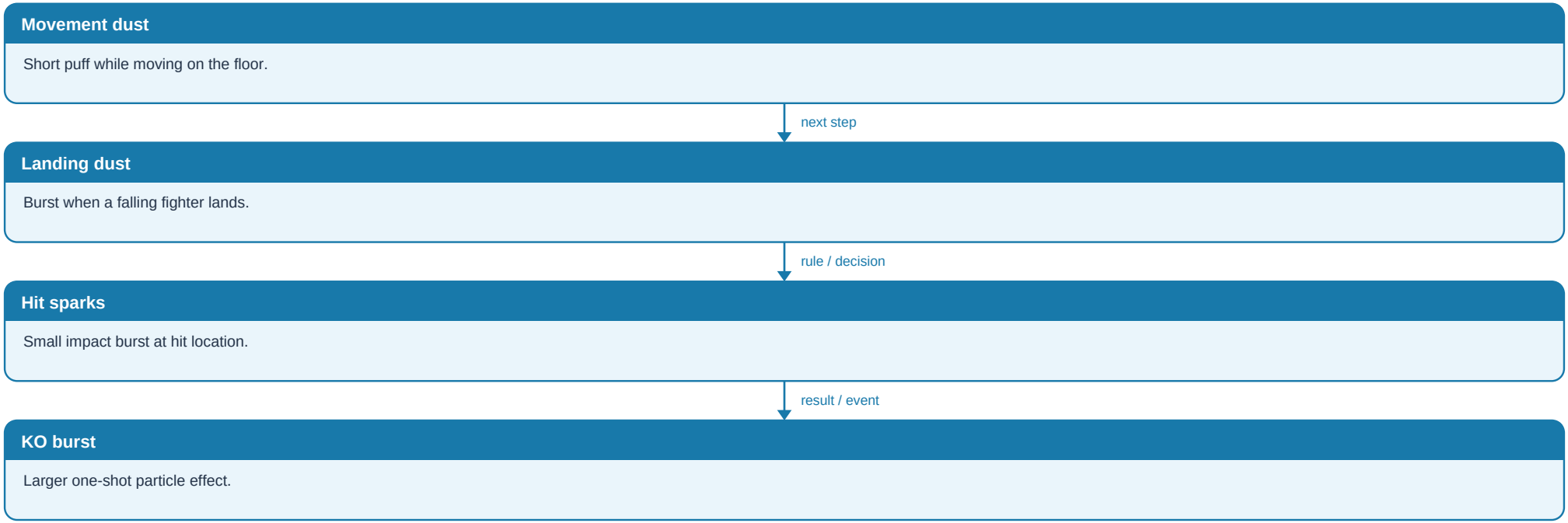
Separate camera presentation from fighter physics: zoom does not change the game world rules.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

25. Particle feedback

Small particles make movement, attacks, and knockouts easier to read.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Particle bursts are created with CPUParticles2D using a generated radial texture. They are spawned at positions sent by fighter events.

HOW THE FLOW WORKS

Effects are presentation output. They do not collide with fighters or apply damage.

PRESENTER NOTE

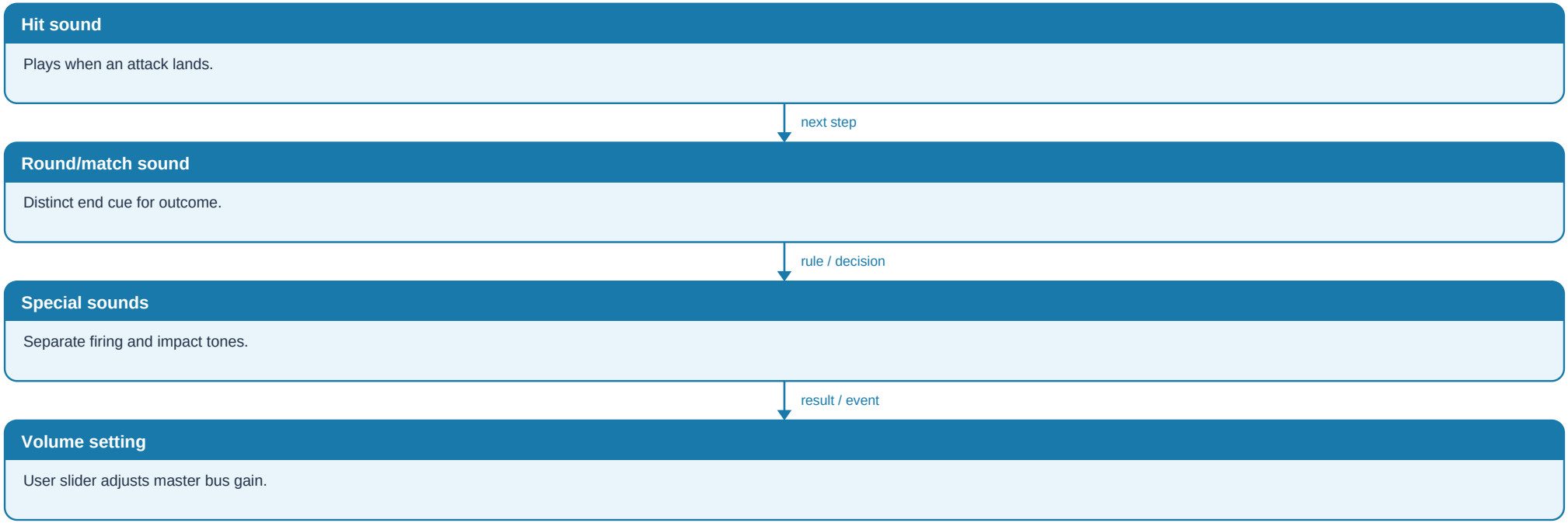
Explain that hit effects complement existing feedback such as flash and camera shake.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

26. Sound effects and master volume

The project creates simple generated tones for feedback and routes them through a master bus.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Audio clips are generated in code to keep the prototype free of external sound assets. Different tone sequences distinguish normal hits, round end, special firing, and special impact.

HOW THE FLOW WORKS

The master volume slider changes output level globally. A sound cue is triggered by the event, not by the HUD displaying text.

PRESENTER NOTE

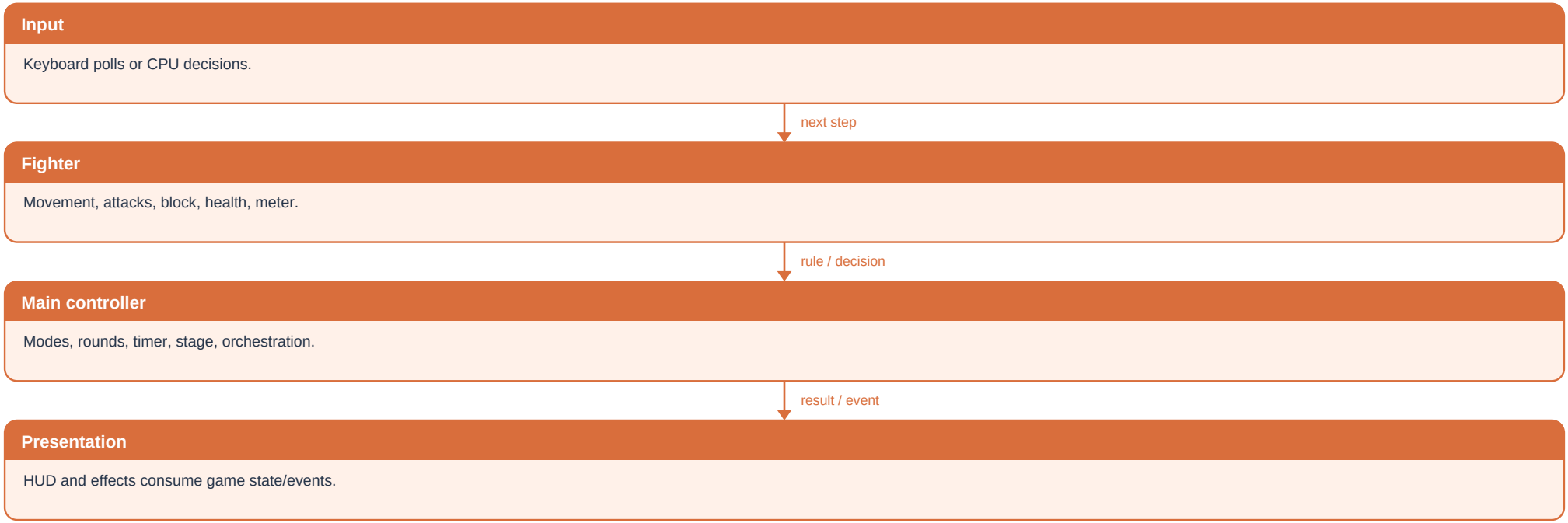
If asked about assets, note that the effects are synthesized at runtime from waveform samples.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

27. Backend boundary: local Godot logic, no server

All match rules run on the same computer; there is no network service or remote game server.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

This project is a local/offline prototype. Its backend is the logic layer inside the running Godot application, not a web API or cloud service.

HOW THE FLOW WORKS

main.gd coordinates match-level systems. fighter.gd owns each fighter's body and combat behavior. Signals communicate state changes to the controller and display layer.

PRESENTER NOTE

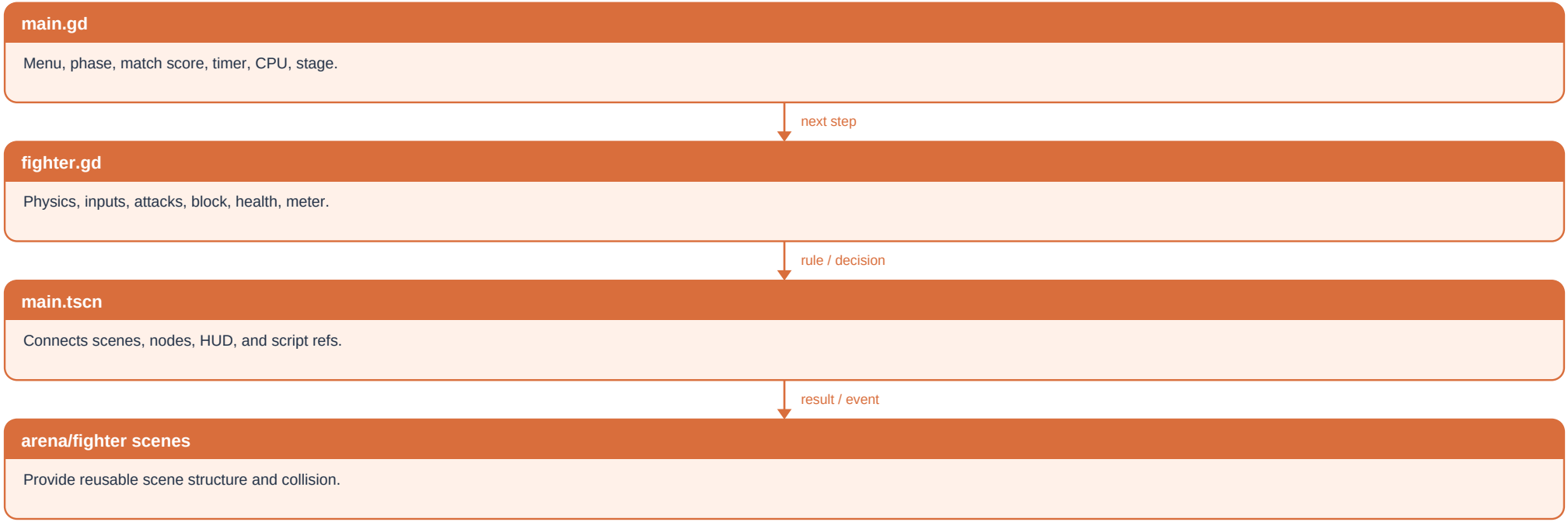
Use this clarification if the teacher asks where the backend server is: there is none because the game is local-only.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd | scenes/game/main.tscn

28. Responsibilities: main.gd versus fighter.gd

The code is divided by scope: match-wide orchestration and per-character behavior.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

A useful ownership rule: if behavior concerns both fighters or the whole match, it tends to belong to main.gd. If it is a single fighter's immediate movement or hit response, it belongs to fighter.gd.

HOW THE FLOW WORKS

The boundary is cooperative rather than completely isolated: main.gd sets flags and connects signals; fighter.gd emits events back to it.

PRESENTER NOTE

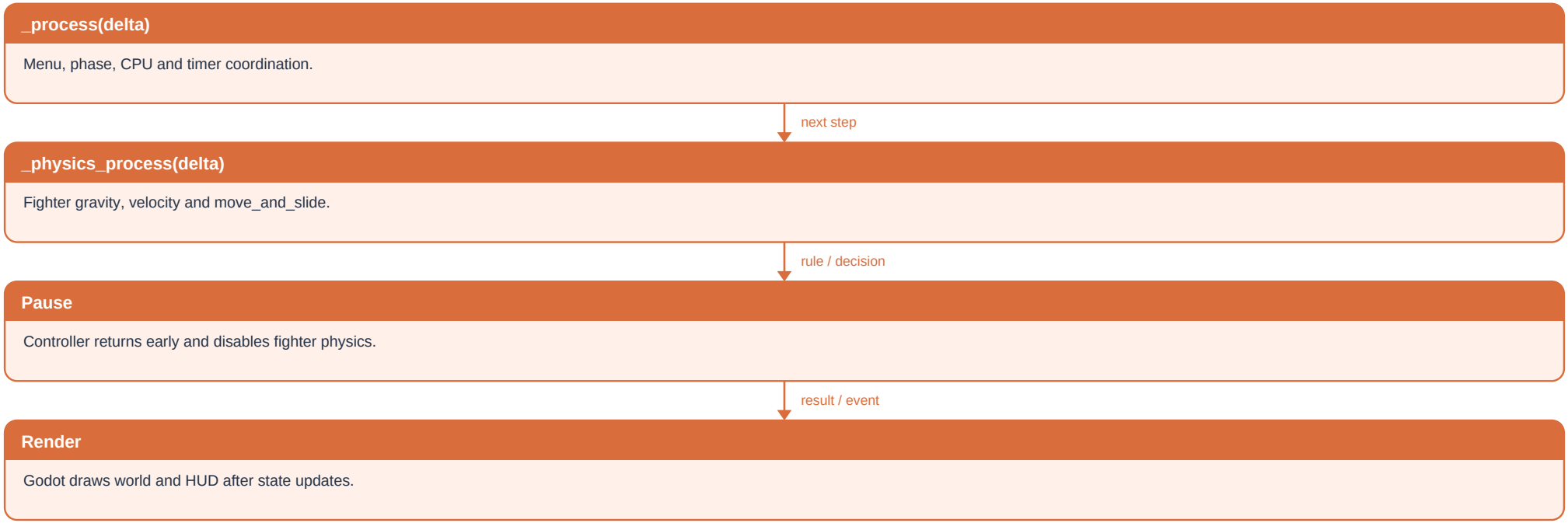
Show both scripts and explain one function from each to illustrate the split.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd | scenes/game/main.tscn

29. Frame lifecycle: input, process, and physics

The project separates match/menu updates from fixed-step character physics.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Frame callbacks divide responsibilities. The controller's general process handles menu edges, phase timing, and camera work; fighters use physics processing for movement and collisions.

HOW THE FLOW WORKS

Pausing halts normal simulation and particle speed. Resuming restores the prior phase and fighter physics.

PRESENTER NOTE

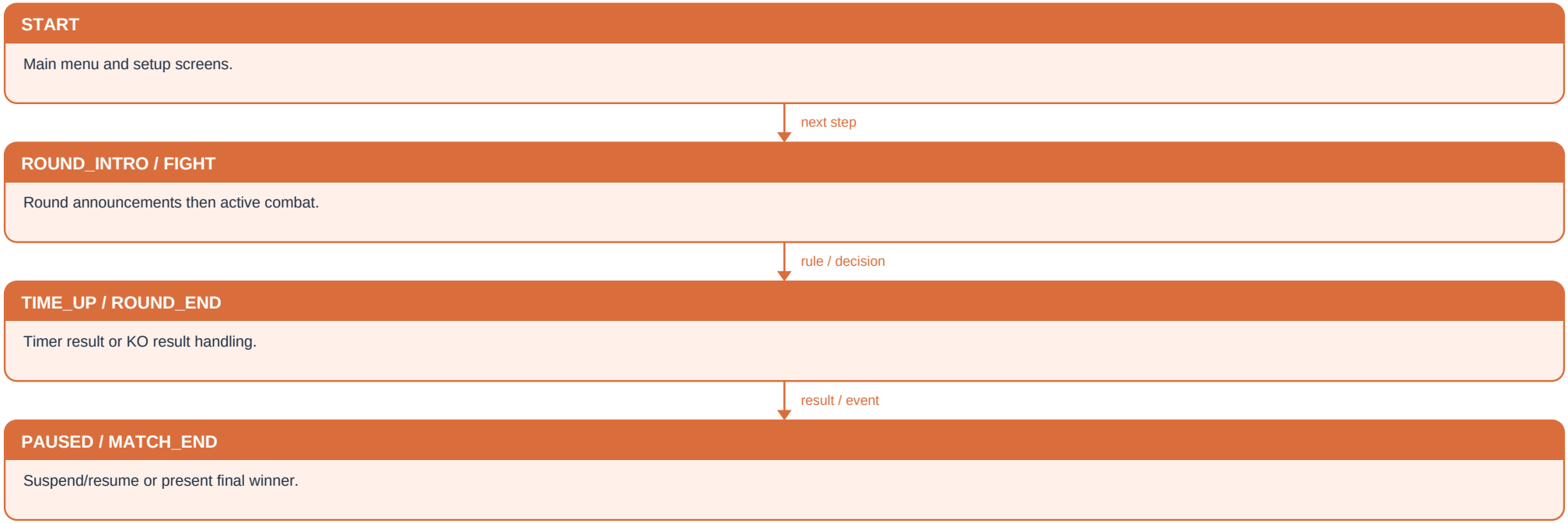
Explain why physics belongs in the physics callback: collision movement is handled consistently by Godot's physics step.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

30. Main phase state machine

The phase variable determines whether the game is on a menu, in a round, paused, or finished.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The phase enum is the central match-state guard. Controller logic updates the phase and related timers/messages; fighter `can_fight` gates whether input may affect a character.

HOW THE FLOW WORKS

Transitions happen after selecting a mode, during round intro, when a fighter is defeated, when time reaches zero, or when Escape toggles pause.

PRESENTER NOTE

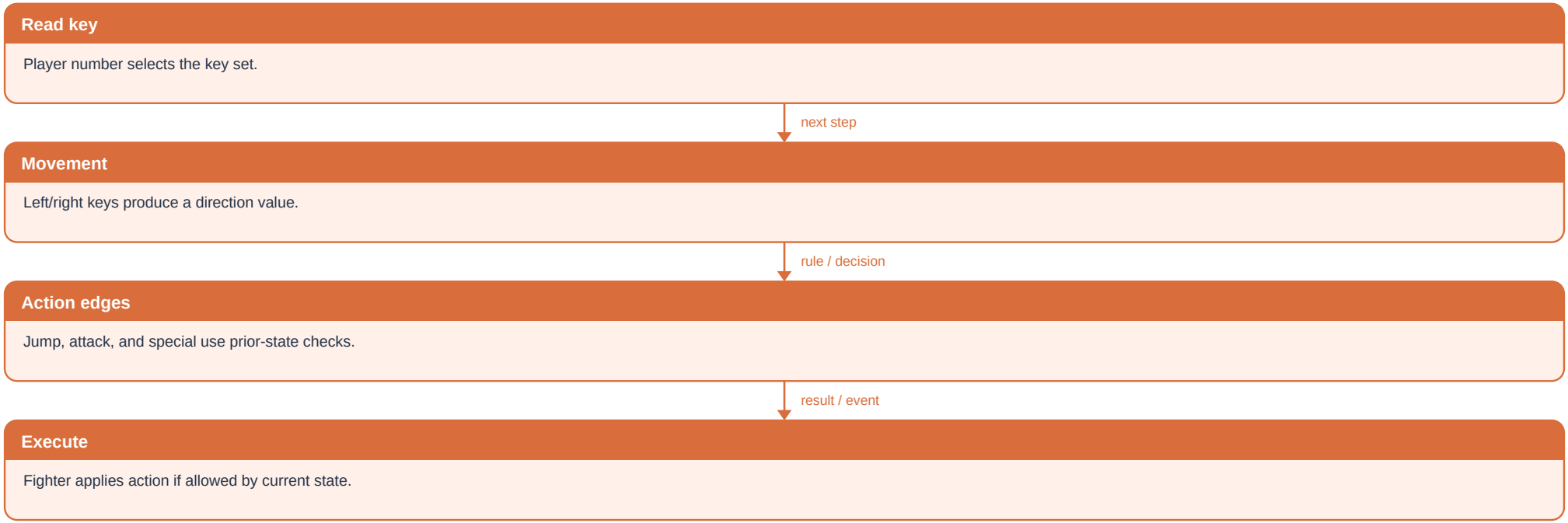
A phase machine is easier to reason about than scattered booleans because each frame can check a named state.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

31. Human keyboard input to fighter actions

Each fighter converts the shared keyboard into movement and action requests.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Player 1 and Player 2 are handled by the same Fighter methods with different keys. Movement input returns a direction; action requests are checked against cooldown, meter, and fight state.

HOW THE FLOW WORKS

Block is held-state input. Attack and jump are edge-detected so holding a key does not repeatedly create new actions.

PRESENTER NOTE

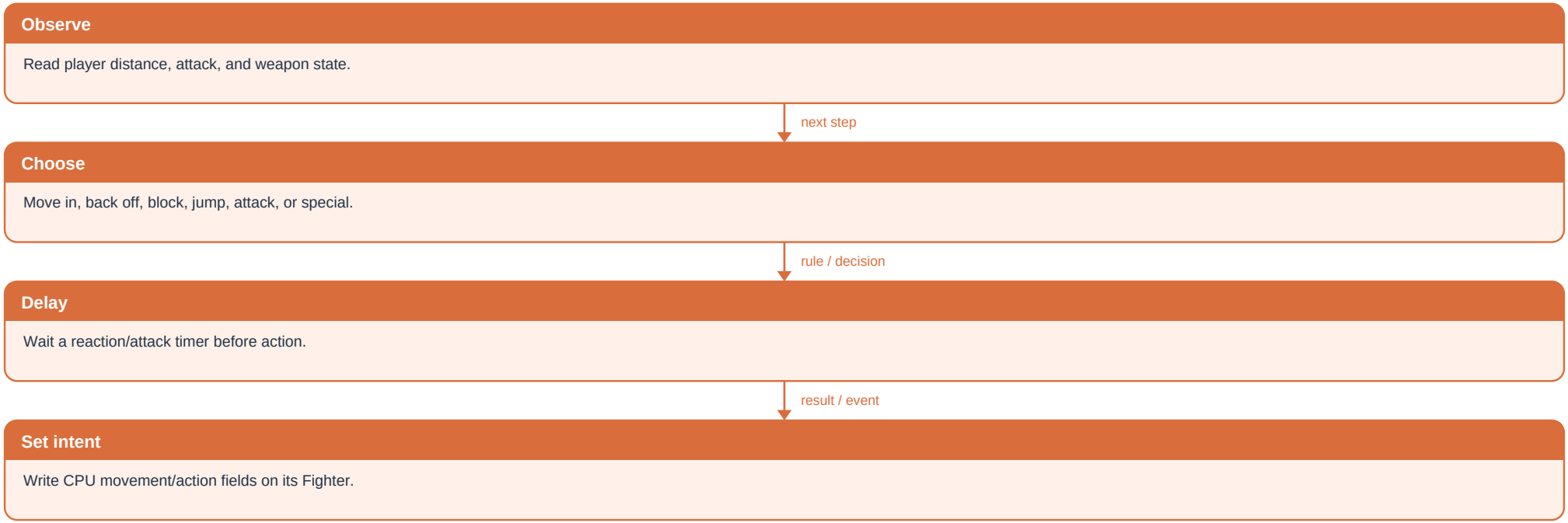
Trace one key: A moves Player 1 left; then explain where facing is recomputed.

PROJECT FILES scripts/player/fighter.gd | project.godot

32. CPU decision pipeline

The CPU is a controller-generated source of intent that uses the same fighter action methods.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The CPU is not a separate physics entity. main.gd decides actions and sets fields such as cpu_move_direction and cpu_attack_pressed on Player 2.

HOW THE FLOW WORKS

fighter.gd then reads those CPU fields through the same input adapter used by human control. This reuses cooldown, damage, blocking, and hit rules.

PRESENTER NOTE

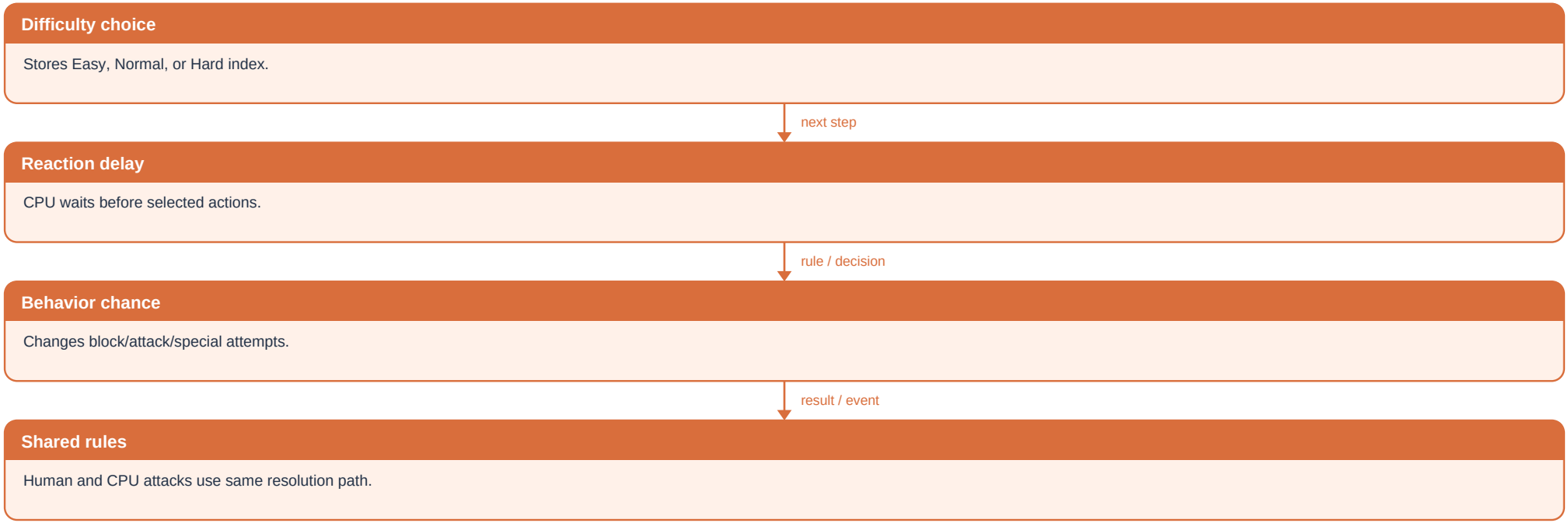
Explain that the CPU does not bypass normal combat damage logic.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

33. CPU difficulty and fairness controls

Difficulty varies CPU response probability and timing rather than changing fighter damage.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The project defines difficulty arrays for reaction delay, block likelihood, attack frequency, and special use. Randomized delays prevent perfectly synchronized responses.

HOW THE FLOW WORKS

Damage values and the fighter's normal cooldown are properties of the combat system; difficulty changes how often or how quickly CPU intent is issued.

PRESENTER NOTE

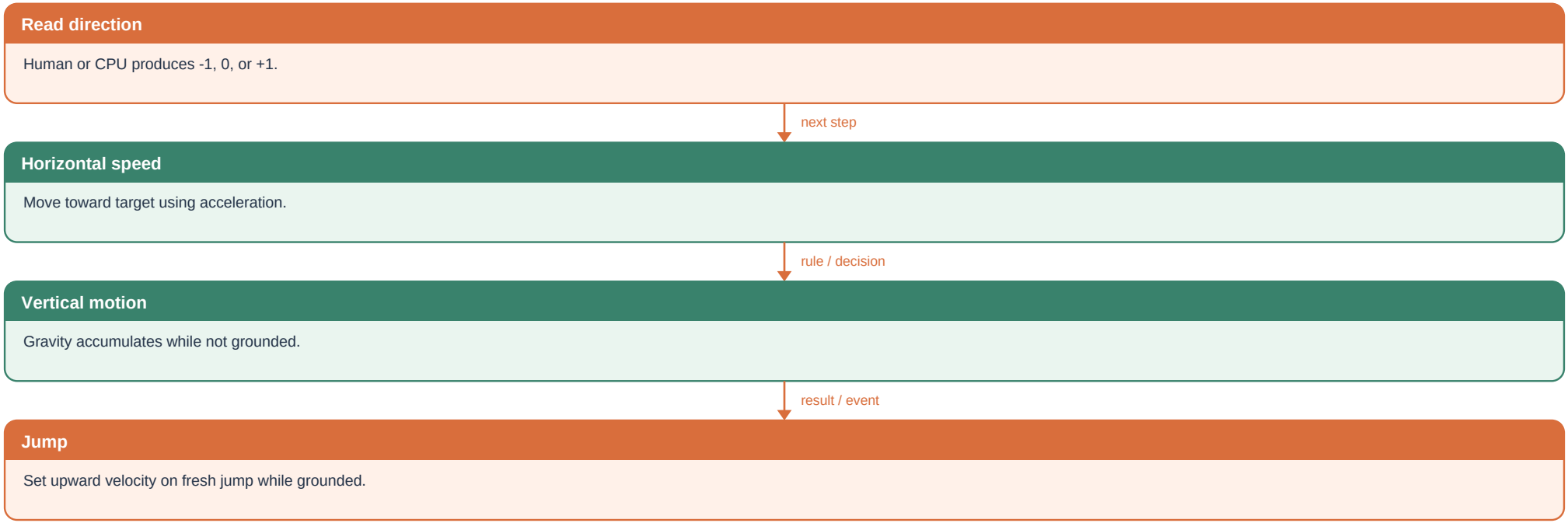
When describing fairness, focus on imperfect reactions and shared fighter rules.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

34. Movement, acceleration, gravity, and jump

fighter.gd updates velocity and lets Godot resolve movement against colliders.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Horizontal motion uses move_toward toward a target speed. When not fighting or when hitstunned, horizontal velocity moves back toward zero.

HOW THE FLOW WORKS

The physics body then calls move_and_slide, after which bounds or Arena 2 hazard checks can be applied.

PRESENTER NOTE

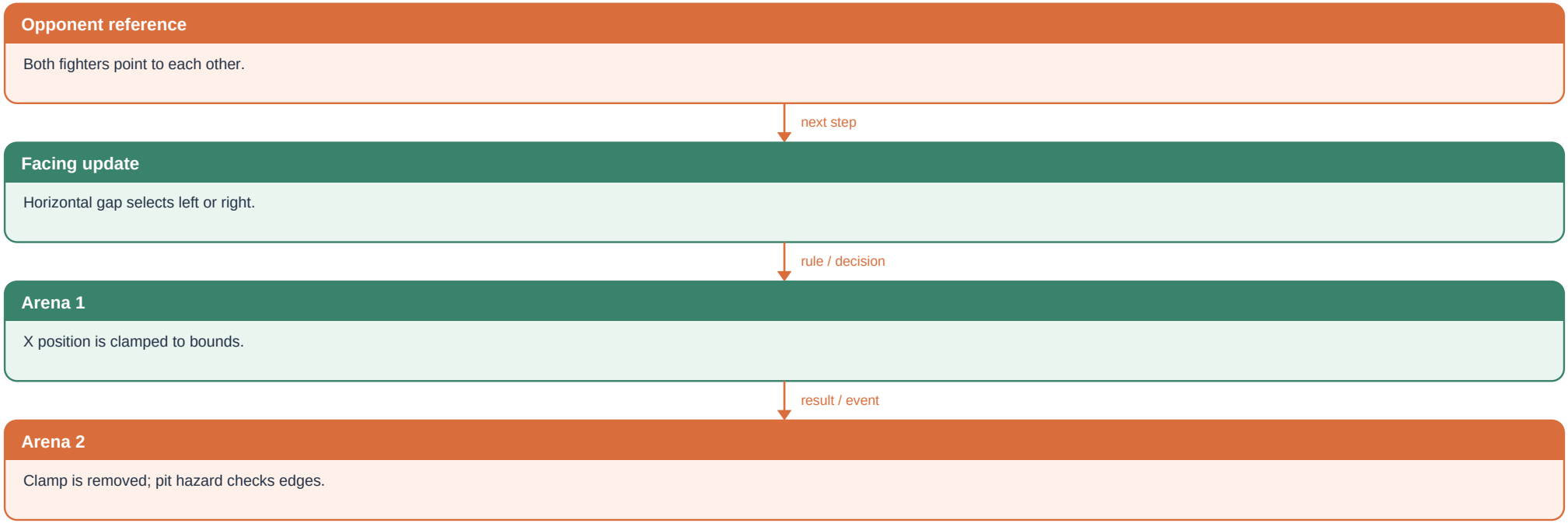
Current constants include move speed 300, acceleration 2200, gravity 1400, and jump velocity -560.

PROJECT FILES scripts/player/fighter.gd | scenes/player/fighter.tscn

35. Facing, floor collision, and arena boundaries

Facing direction powers attack direction checks and is recomputed from opponent position.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The visual child is mirrored by setting its horizontal scale. The body remains in world coordinates, allowing attacks to compare positions with the opponent.

HOW THE FLOW WORKS

Arena 1 keeps the player inside full-width limits. Arena 2 lets them cross the center platform edge so falling into lava is possible.

PRESENTER NOTE

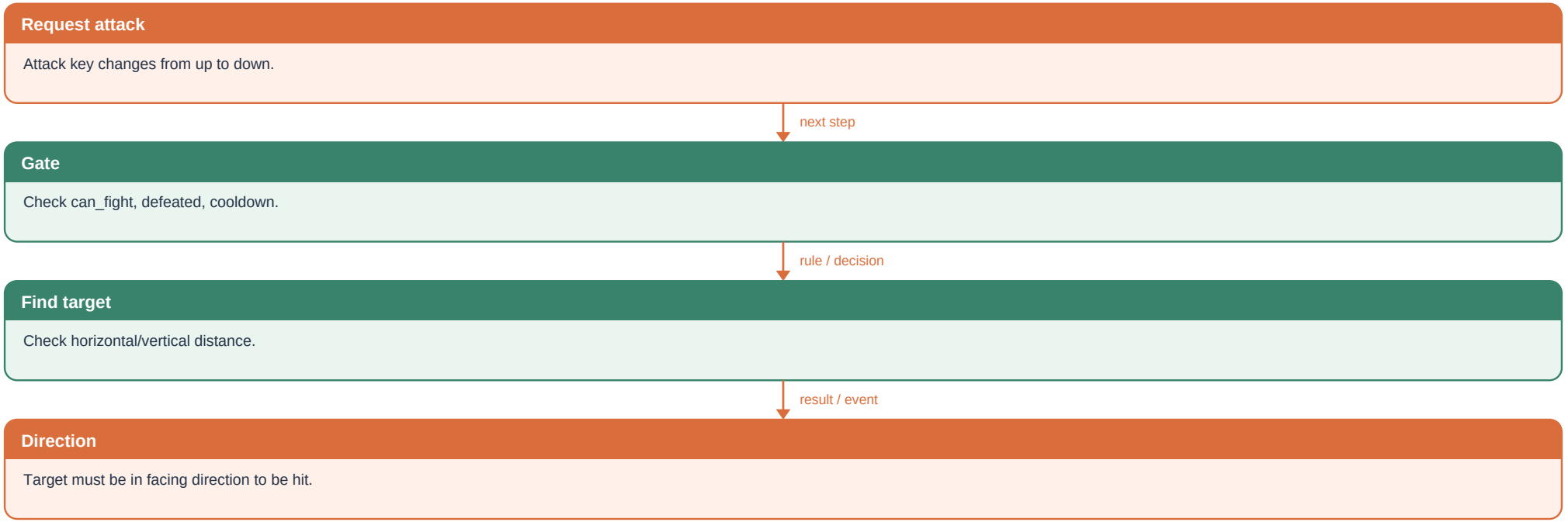
Explain that auto-facing preserves the existing forward-only attack check.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

36. Attack cooldown, range, and facing check

A punch begins only if the fighter can act and its cooldown has reached zero.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The attack is a code-level range check rather than a separate animated hitbox scene. It scans the fighters group and checks opponent positions.

HOW THE FLOW WORKS

Unarmed range is 90; armed range is 140. The cooldown is 0.45 seconds, applied when the attack is initiated.

PRESENTER NOTE

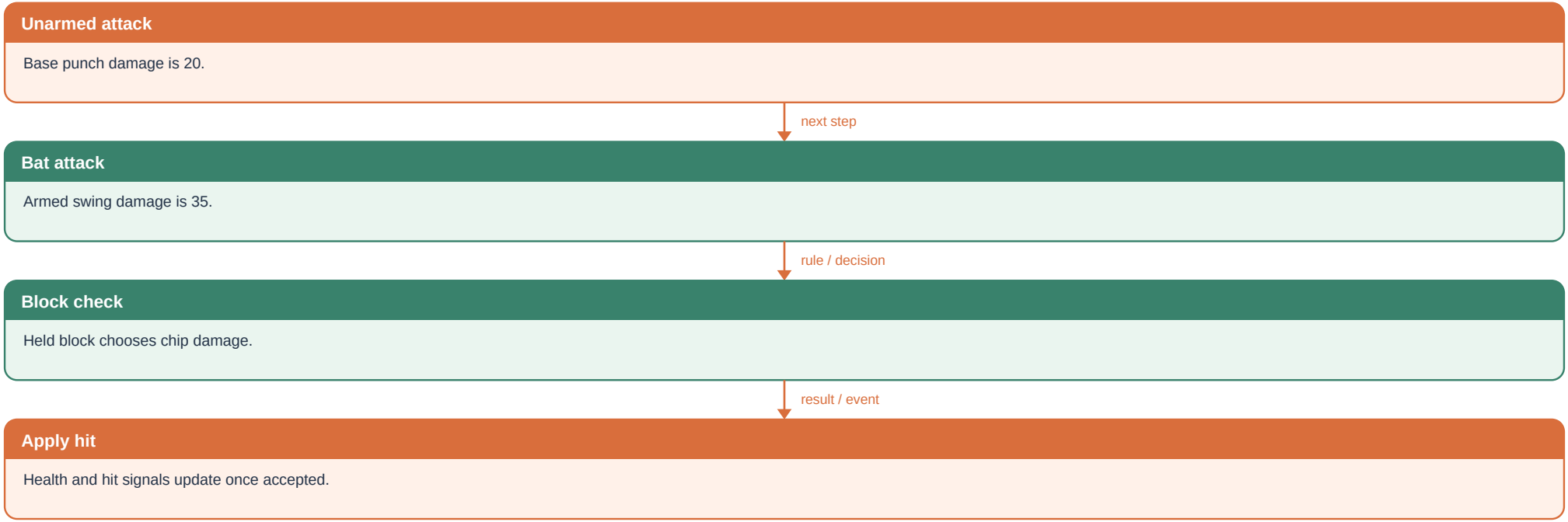
The visual arm swing is feedback; the range test decides whether damage lands.

PROJECT FILES scripts/player/fighter.gd

37. Damage choice and block resolution

The attacker selects damage based on weapon state; defender applies block rules.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The damage amount is selected by attacker state. The defender checks whether blocking is active and reduces ordinary punch/bat attacks to chip damage.

HOW THE FLOW WORKS

A blocked punch/bat hit applies 5 damage and no hit knockback. The special projectile has its own 40 normal / 10 blocked damage path.

PRESENTER NOTE

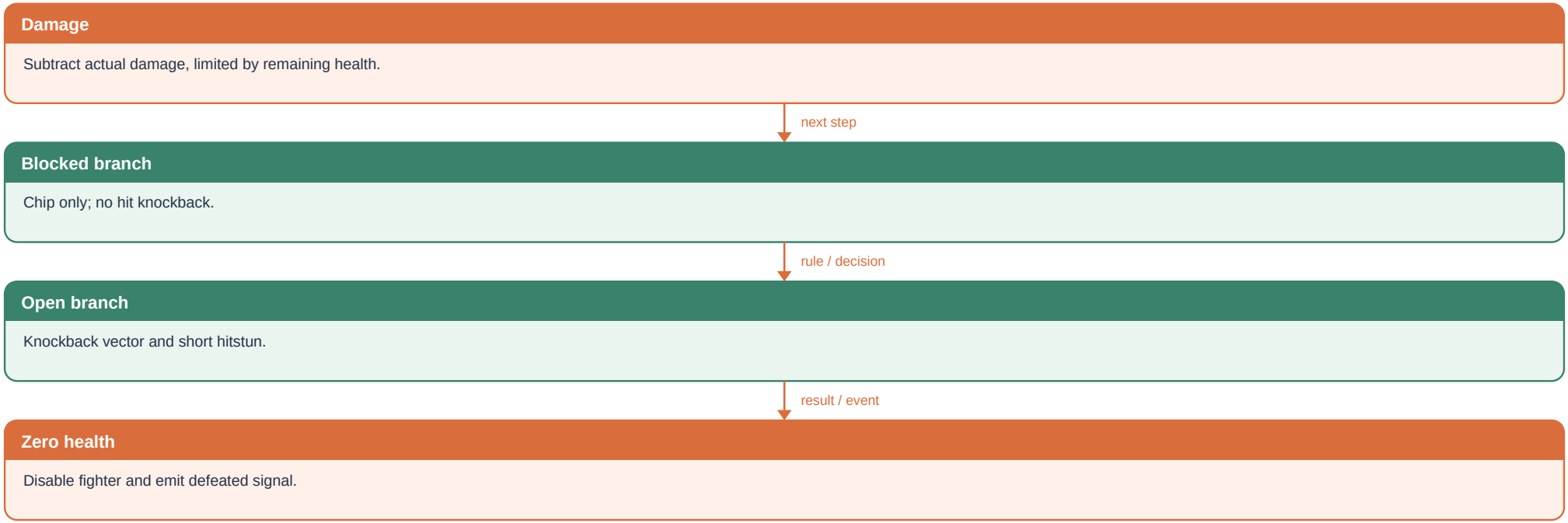
Keep special damage separate from normal attack values when explaining rules.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

38. Hitstun, knockback, flash, and defeat

An accepted hit triggers gameplay response and presentation events.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

An unblocked normal hit applies horizontal knockback 380 and vertical -180, plus hitstun. A special uses stronger knockback constants.

HOW THE FLOW WORKS

Both branches can disarm a weapon holder. The visual hit signal drives flash, particle burst, shake, and damage number display.

PRESENTER NOTE

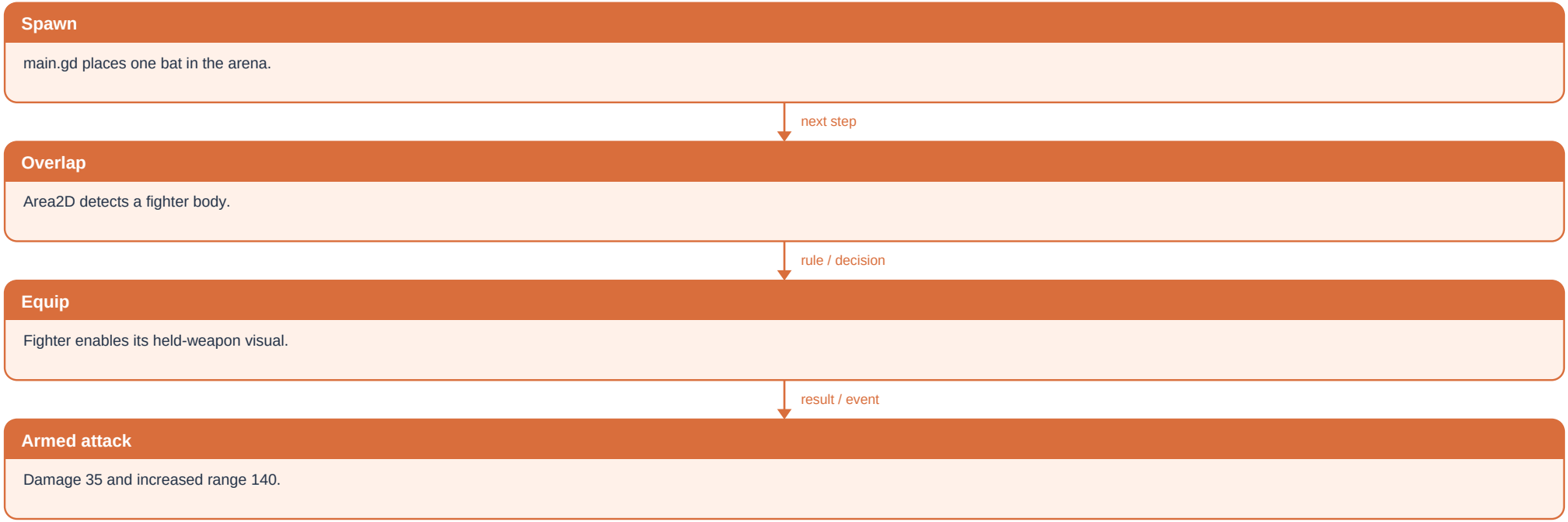
Health reaching zero disables further fighter action and notifies main.gd to end the round outside Practice Mode.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

39. Bat pickup, carrying, and armed attack

The bat is a world pickup that changes fighter state after overlap.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The WeaponPickup Area2D exists in main.tscn. main.gd checks overlapping bodies during an active fight and equips the first eligible fighter.

HOW THE FLOW WORKS

The visual moves into the fighter's hand. A HUD indicator reports the holder. Attack cooldown remains the same as normal.

PRESENTER NOTE

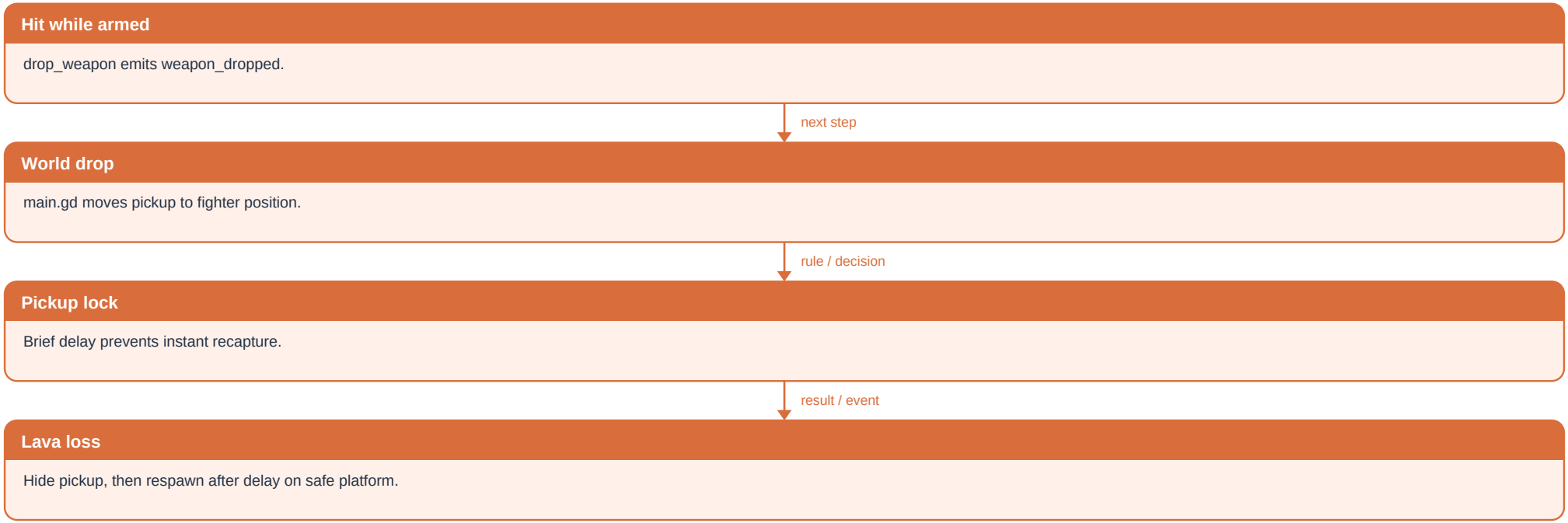
Describe pickup as state transfer: floor object hidden, Fighter.has_weapon becomes true.

PROJECT FILES scenes/game/main.tscn | scripts/game/main.gd | scripts/player/fighter.gd

40. Weapon drop, pickup lock, and lava respawn

A hit forces an armed fighter to drop the bat; Arena 2 also handles the bat falling into lava.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

main.gd tracks the holder, falling state, pickup delay, and lava-respawn timer. The recent dropper must leave the overlap before taking the weapon again.

HOW THE FLOW WORKS

The bat is lost when it falls beyond the safe platform in Arena 2. Arena 1 uses the same pickup and combat system without the pit loss rule.

PRESENTER NOTE

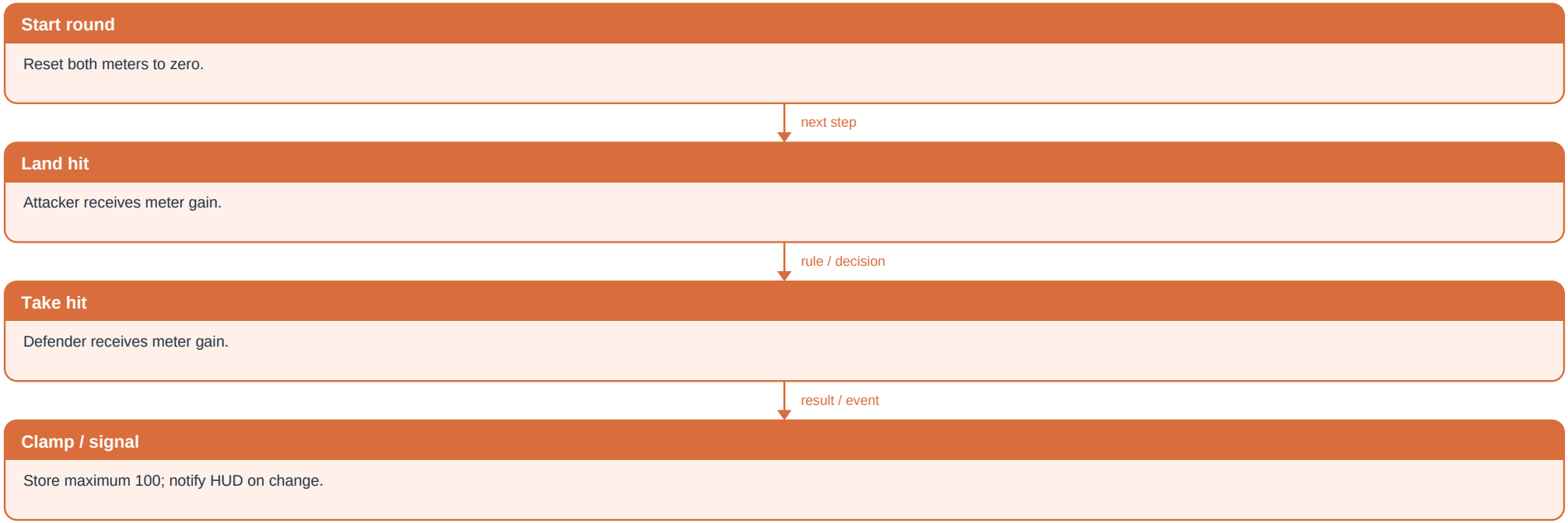
Mention the separation between fighter's equip/drop signal and the world pickup manager.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd | scenes/game/main.tscn

41. Super-meter gain, cap, and reset

Meter is stored per fighter and clamped between zero and one hundred.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The current script adds 25 on a landed hit and 25 when taking a hit, then clamps to a maximum of 100. A meter signal updates the displayed bar.

HOW THE FLOW WORKS

These values describe the current published code; they are separate from normal punch, bat, and chip damage values.

PRESENTER NOTE

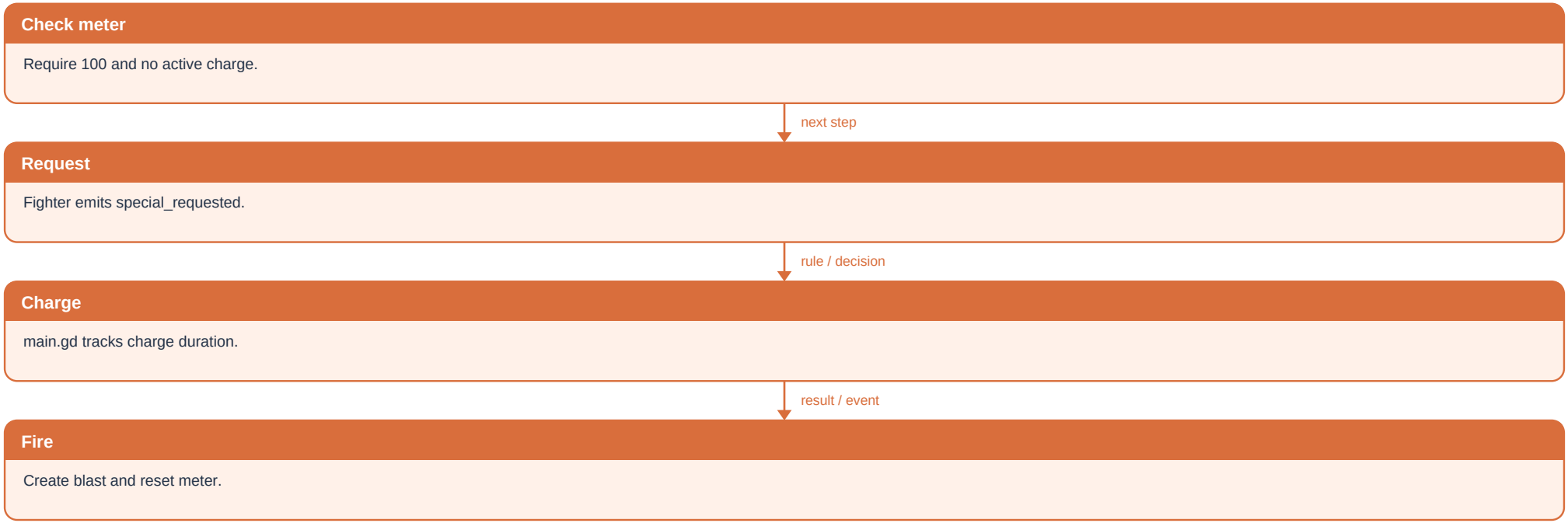
Explain that a full meter is an eligibility condition for the special, not a damage multiplier.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

42. Special request and charge-up phase

A full meter creates a special request, followed by a brief charge pose.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The charge state changes the fighter pose and displays a small glowing orb. The controller owns the special timing and projectile spawn.

HOW THE FLOW WORKS

CPU special requests use the same request path after the CPU's reaction/difficulty checks. The meter resets when firing.

PRESENTER NOTE

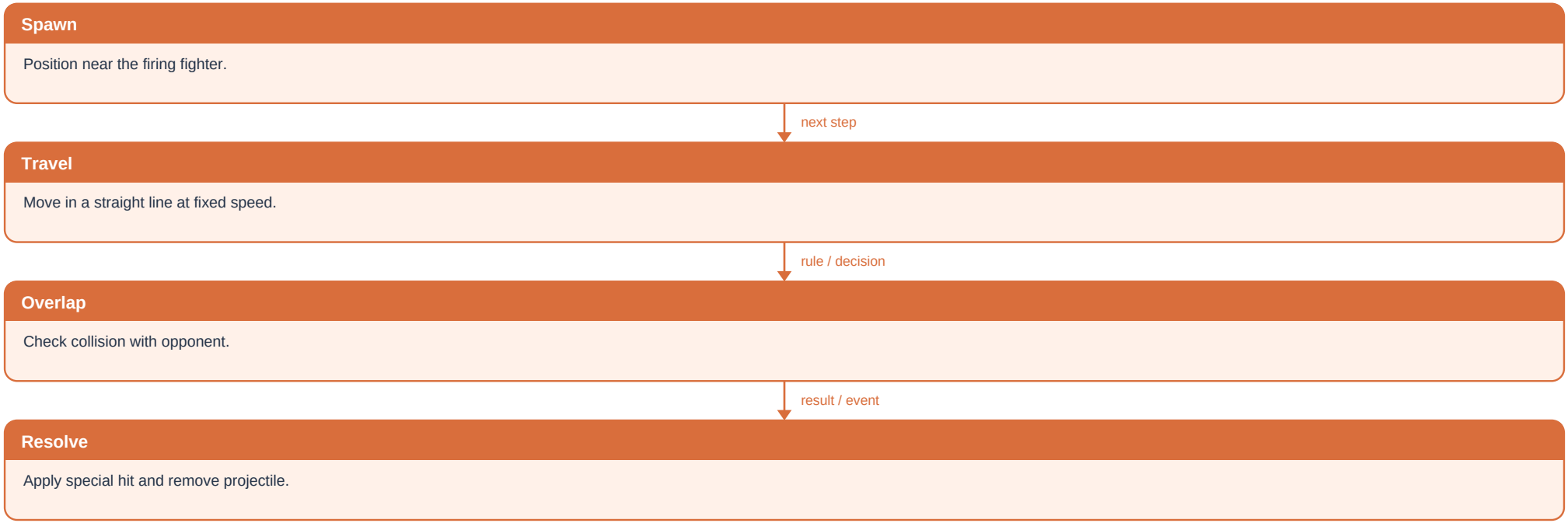
Show charge as a telegraph before the blast, not as extra damage over time.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

43. Special projectile travel and hit

The blast travels horizontally in the firing direction and resolves on contact.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Special damage is 40 unblocked and 10 on block. Strong knockback is applied only if the defender is not blocking.

HOW THE FLOW WORKS

Only one blast per fighter is allowed on screen. The projectile is removed when it hits a fighter or leaves arena bounds.

PRESENTER NOTE

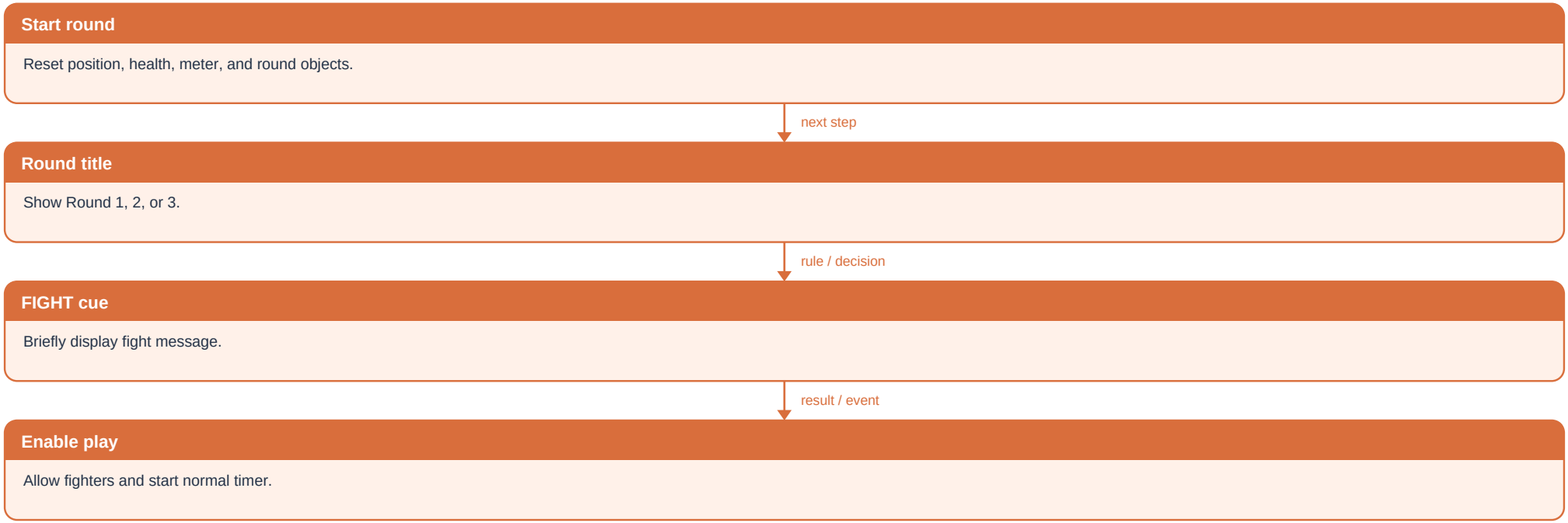
Explain that this is a local Area2D projectile rather than an online/networked event.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

44. Round intro and FIGHT timer start

Every round resets fighters and presents an intro before combat input is enabled.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The phase machine holds fighters inactive during intro. The timer begins with the FIGHT cue rather than while the intro is displayed.

HOW THE FLOW WORKS

Practice mode uses the round setup but skips the normal countdown and win condition.

PRESENTER NOTE

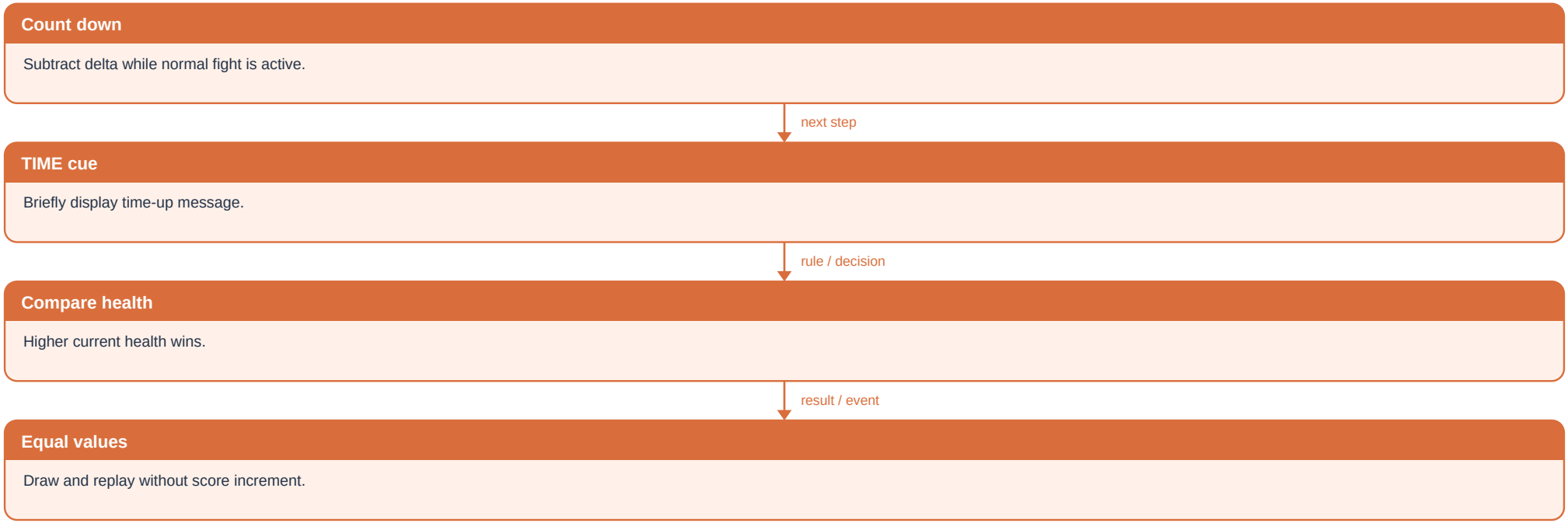
This sequence ensures every round starts from a clean, readable state.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

45. Countdown expiry and draw handling

At zero seconds the timer stops and remaining health determines the round result.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

During the last ten seconds, the timer changes color and pulses. Pause, KO, and match-end stop the timer.

HOW THE FLOW WORKS

On equal health, the phase becomes ROUND_DRAW; after its short display, start_round runs again without increasing either round-win count.

PRESENTER NOTE

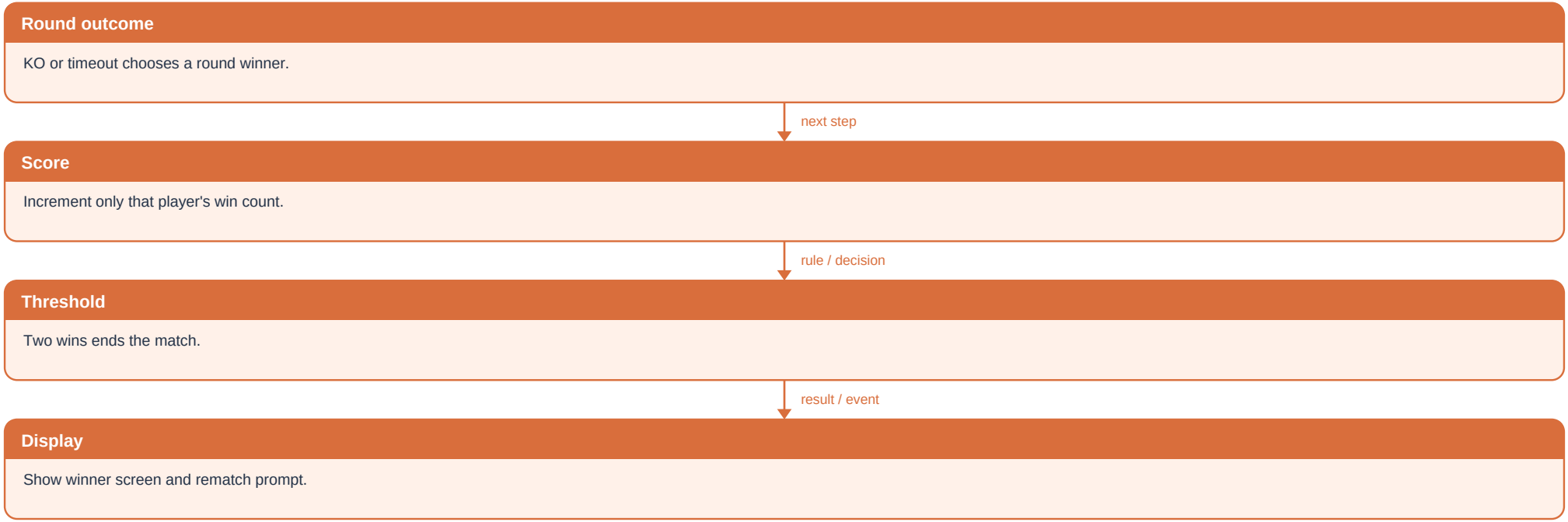
A draw is a replay, not a tie-break score or a win for either player.

PROJECT FILES scripts/game/main.gd

46. Best-of-three scoring and match victory

Normal matches end when one side reaches two round wins.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The system tracks round_number and two separate round-win counts. Win indicators are refreshed after each result.

HOW THE FLOW WORKS

A draw does not add a win. Practice mode bypasses round victory and match-end handling.

PRESENTER NOTE

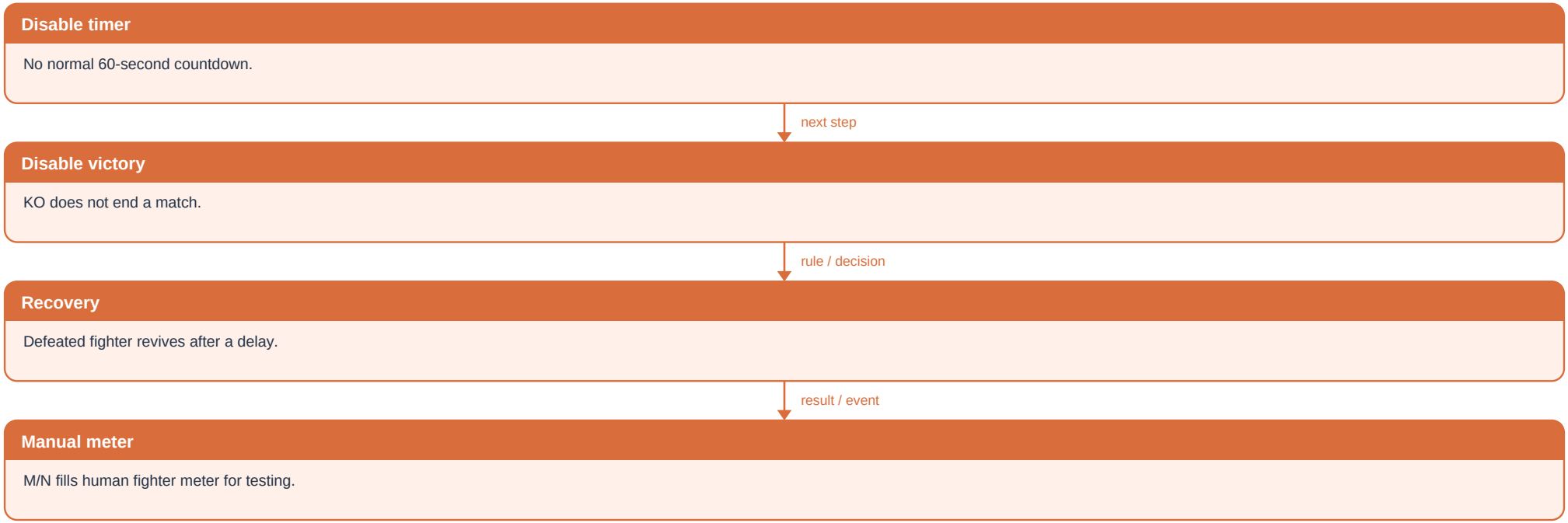
Explain the rule as first-to-two, maximum three rounds when there are no draws.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

47. Practice Mode: no clock, recovery, and meter test

Practice changes match flow while retaining core fighter interactions.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The controller handles Practice recovery timers and avoids normal match-ending behavior. Fighters retain movement, block, attack, weapons, and special move logic.

HOW THE FLOW WORKS

Practice can configure a live CPU or passive dummy and can select either stage. Arena 2 hazard behavior still applies.

PRESENTER NOTE

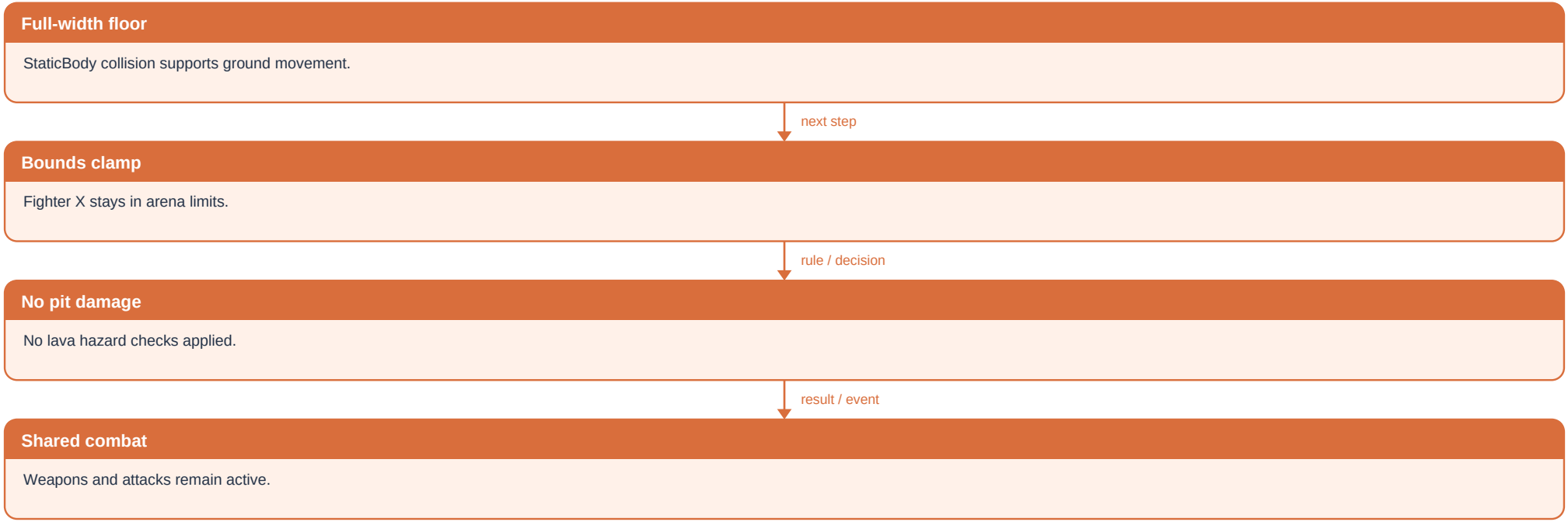
Practice is an alternate ruleset for experimentation, not a separate combat implementation.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

48. Arena 1: standard ground and boundaries

Arena 1 preserves full-width ground and ordinary bounds behavior.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The standard stage is the reference behavior. Stage styling may vary, but Arena 1 does not use the lava damage timer or missing-floor edge rule.

HOW THE FLOW WORKS

The shared Fighter class checks arena_two_enabled before running hazard logic. When disabled, it clamps horizontal position to normal bounds.

PRESENTER NOTE

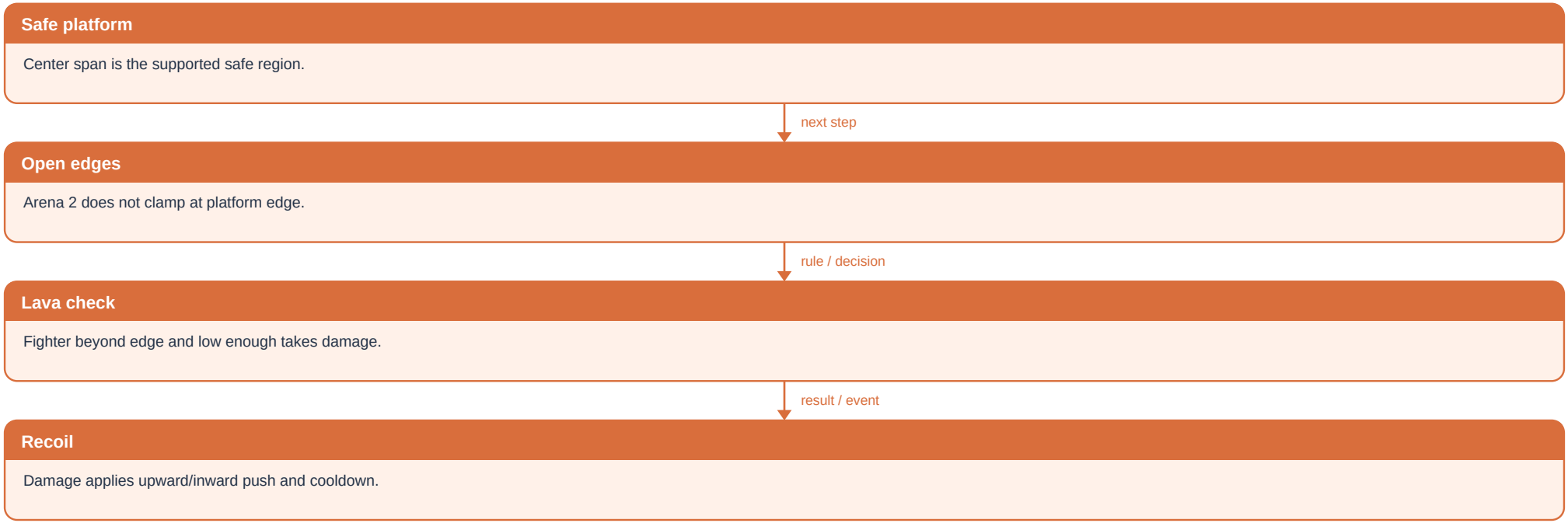
Clarify that Arena 2 does not replace or modify Arena 1 behavior.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd | scenes/arenas/arena.tscn

49. Arena 2: lava hazard and safe platform

Arena 2 changes movement boundaries and introduces periodic damage outside the center platform.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Lava deals 10 damage every 0.5 seconds while the fighter remains in the hazard. The short recoil pushes toward the safe area and prevents immediate repeat damage.

HOW THE FLOW WORKS

The CPU is told to avoid walking into lava, but normal combat knockback can still send it into the pit.

PRESENTER NOTE

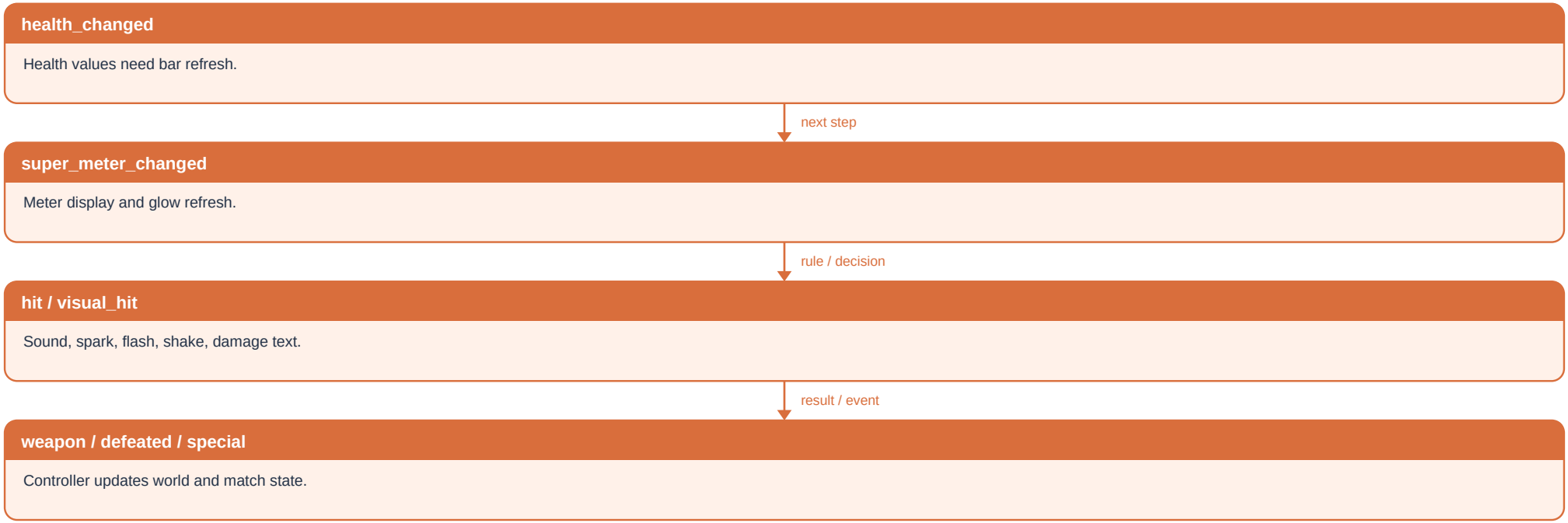
Explain the difference between deliberate CPU path choice and being knocked by the opponent.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

50. Signal and event map

Signals tell main.gd that important fighter state changes occurred.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Signals reduce direct coupling: Fighter owns the action; main.gd responds to the event and updates world-level systems.

HOW THE FLOW WORKS

Signals are connected in main.gd during _ready(). A single hit can emit several related events for health, meter, feedback, and possible weapon drop.

PRESENTER NOTE

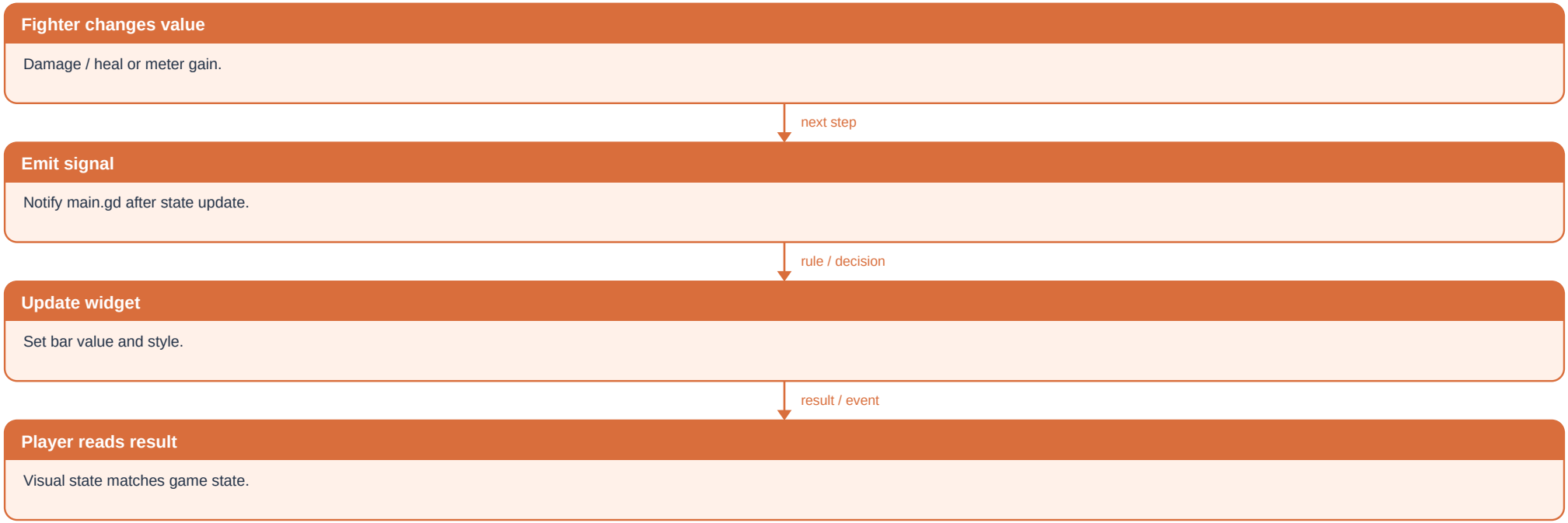
Use one hit as an example of multiple listeners reacting to a single event.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

51. Health and meter update pipeline

The controller relays fighter values to visible ProgressBars.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Bars are outputs of the current model. They should not be used as the authoritative health or super meter store.

HOW THE FLOW WORKS

When a round resets, the controller asks each fighter to reset and their health/meter signals refresh the HUD.

PRESENTER NOTE

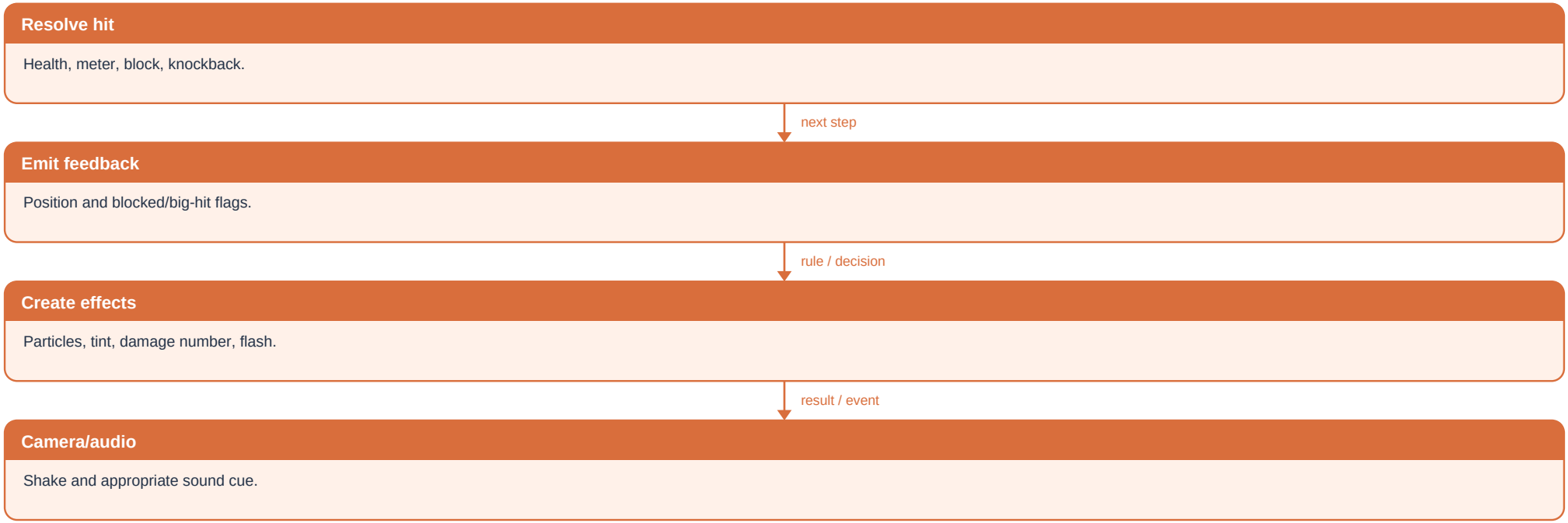
This is a classic model-view split: Fighter is model, ProgressBar is view.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd | scenes/game/main.tscn

52. Hit feedback event pipeline

A landed hit creates gameplay changes followed by layered feedback.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The feedback system is separate from the attack result. Particles, flash, and camera shake communicate impact but do not alter damage values.

HOW THE FLOW WORKS

Blocked hits use a distinct visual tint/spark from unblocked hits. A larger hit may add a short screen tint on top of shake.

PRESENTER NOTE

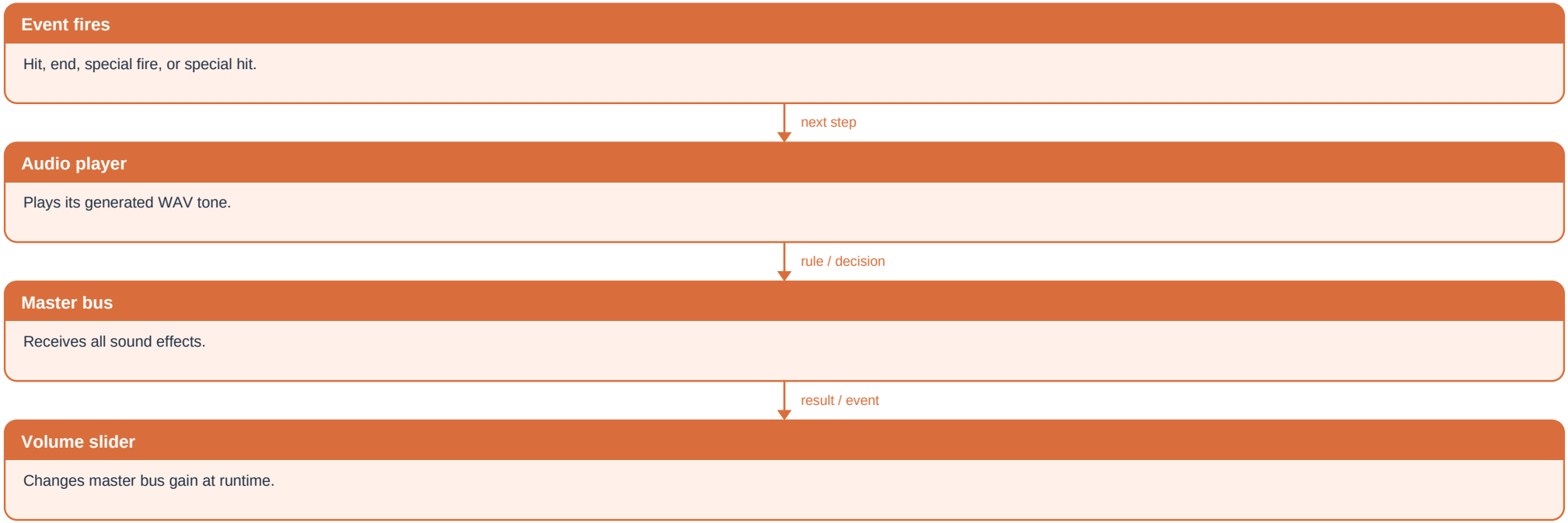
Explain the goal as readability and impact, not extra gameplay strength.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

53. Audio settings to sound output

Generated sound players route their effects through one master volume setting.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The Settings volume option operates at the audio bus level, so it affects all existing effects without changing each effect's gameplay timing.

HOW THE FLOW WORKS

Distinct tone sequences make ordinary impact, round/match end, special fire, and special hit easier to distinguish.

PRESENTER NOTE

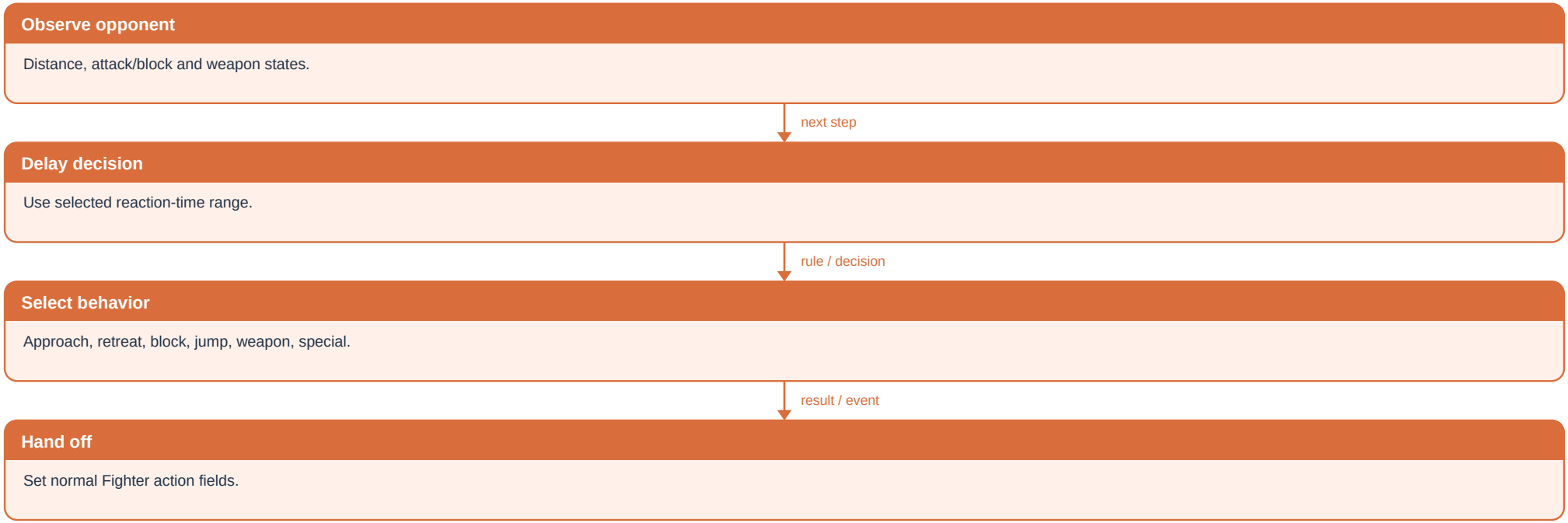
Mention that sounds are synthesized from samples and are not external sound assets.

PROJECT FILES scripts/game/main.gd | scenes/game/main.tscn

54. CPU fairness, reaction delay, and weapon awareness

CPU choices incorporate delay, difficulty, distance, attacks, and pickup state.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The CPU can prioritize a nearby floor weapon, use attacks when in range, and decide when to block or jump. Difficulty changes how readily it tries certain actions.

HOW THE FLOW WORKS

A reaction delay around a fraction of a second gives the human time to act. The CPU's punch, bat, block, and special resolve through Fighter rules.

PRESENTER NOTE

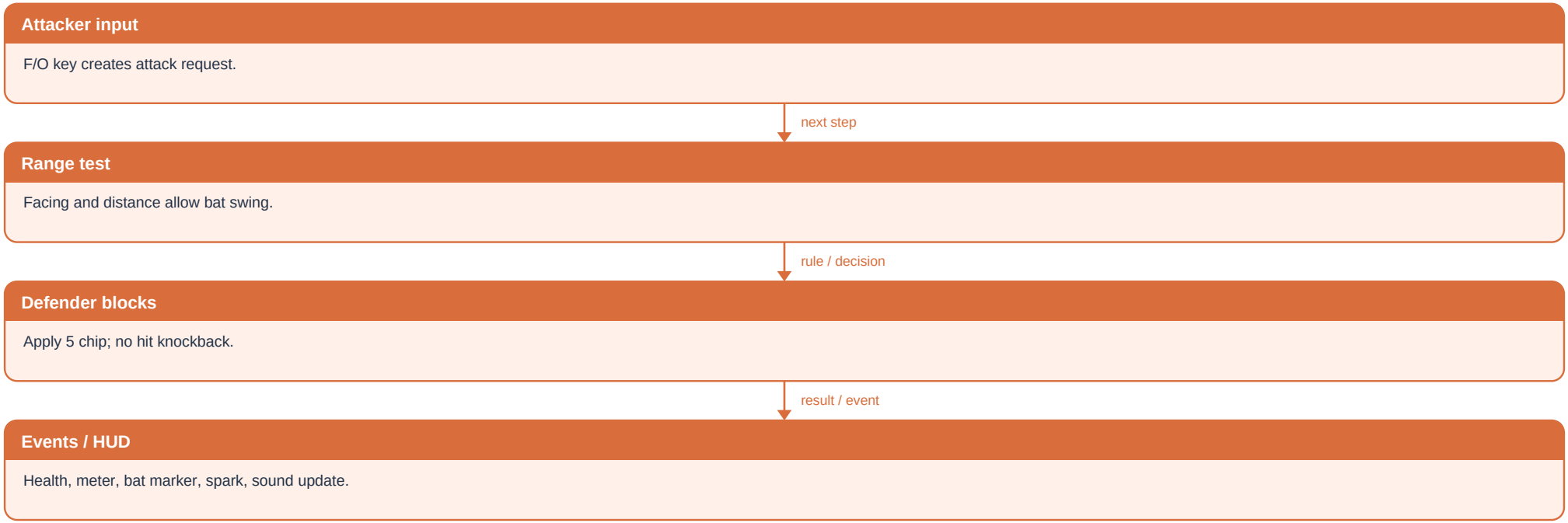
Avoid claiming the CPU makes strategic predictions; it uses simple rules and timers.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

55. End-to-end trace: a blocked bat hit

This trace follows one concrete event across input, combat, signal, and display layers.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The defender loses chip health and gains meter. If the defender held a bat, the hit still disarms them. The attacker's landed hit adds meter and hit feedback plays.

HOW THE FLOW WORKS

The HUD update is downstream of the fighter state change. It cannot cause or prevent the hit.

PRESENTER NOTE

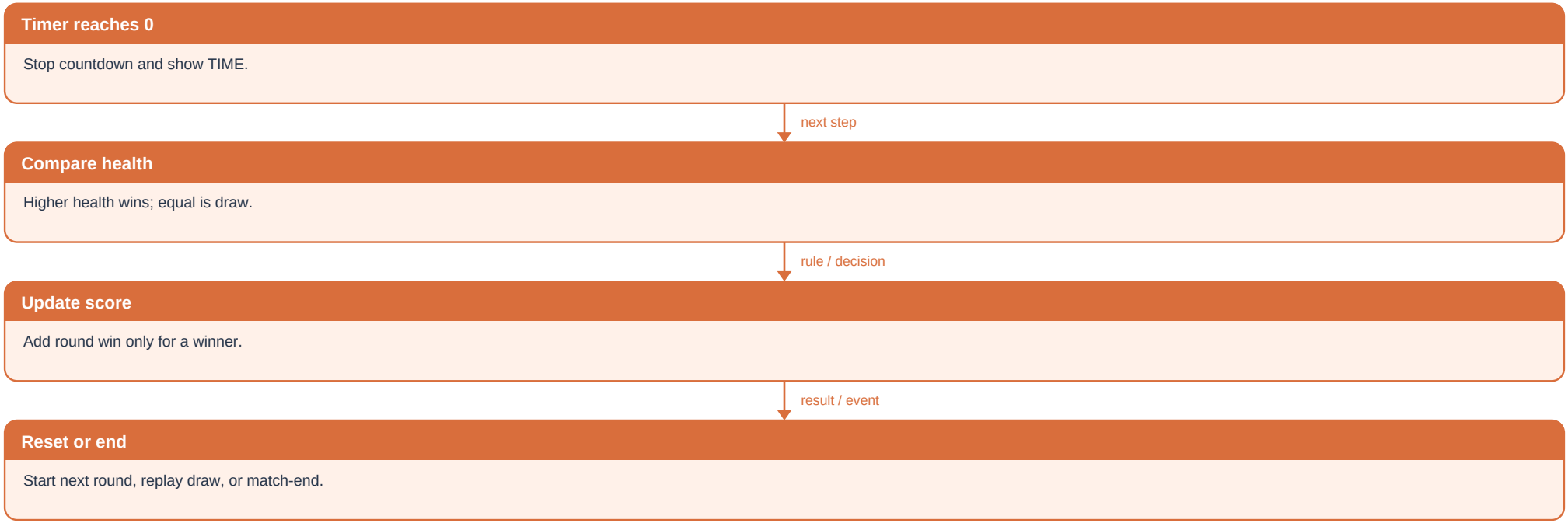
This trace is useful for demonstrating how a single control press involves multiple systems.

PROJECT FILES scripts/player/fighter.gd | scripts/game/main.gd

56. End-to-end trace: timeout and next round

This trace shows how round timing leads to score and round reset.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

The timer is part of the controller phase logic. A timeout result uses current health rather than a separate sudden-death attack.

HOW THE FLOW WORKS

A draw skips score increment. Two accumulated wins end the match; otherwise the next round resets fighters, meter, and round objects.

PRESENTER NOTE

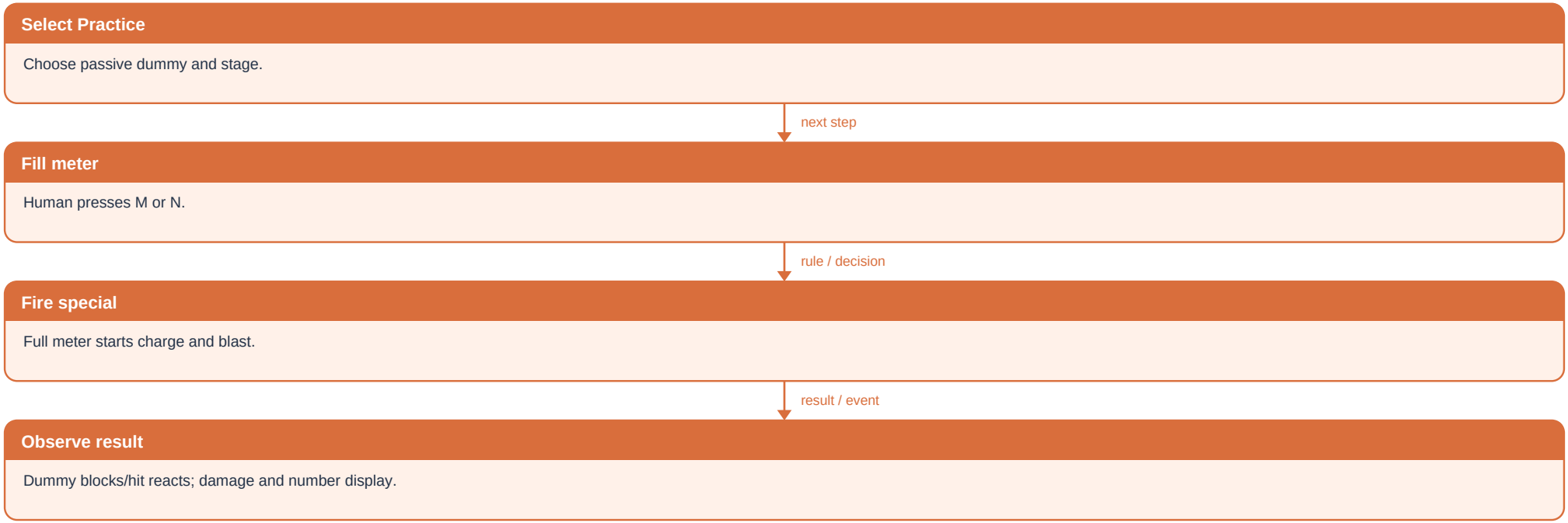
This trace makes the best-of-three rule easy to explain without walking through all code.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

57. End-to-end trace: Practice dummy and super test

This trace follows the training setup from selection to special hit.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Practice skips the normal clock and score result while preserving damage handling. The dummy can be hit and knocked around but does not initiate attacks.

HOW THE FLOW WORKS

A player can use the meter test key to repeat special-move testing without needing to build meter through many hits.

PRESENTER NOTE

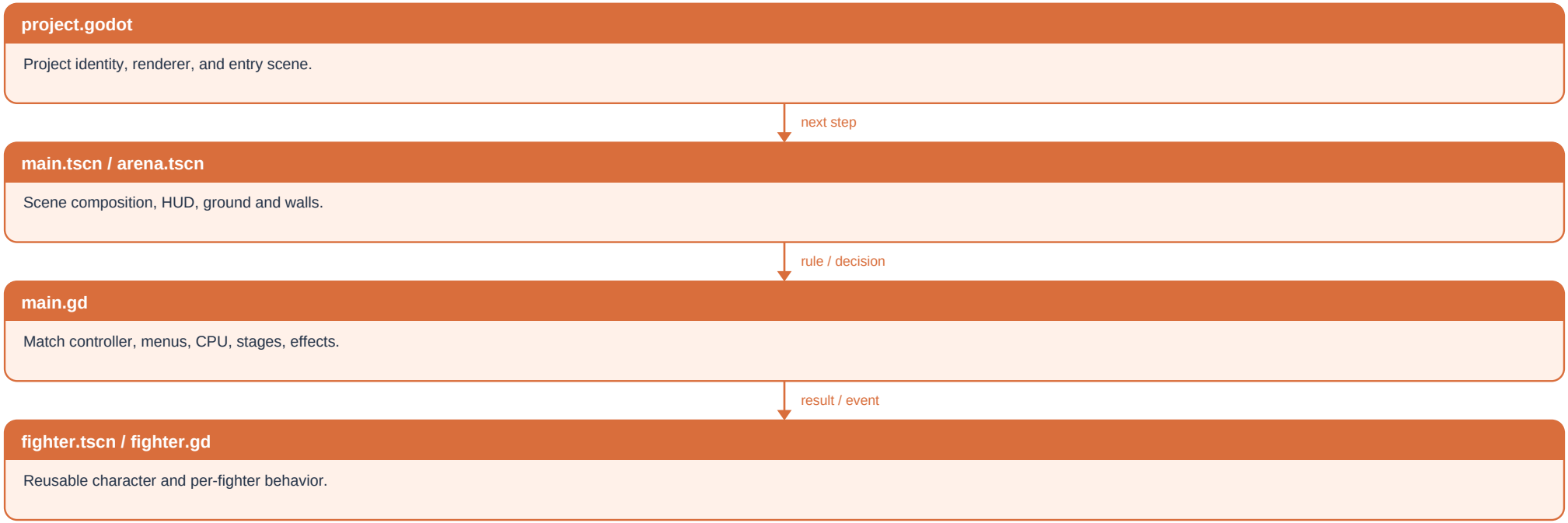
Demonstrate the dummy and special as two distinct practice features.

PROJECT FILES scripts/game/main.gd | scripts/player/fighter.gd

58. File map and glossary for explaining the diagrams

Use this map to connect concepts in the guide to the files in the repository.

FLOW DIAGRAM | follow each arrow from top to bottom



PURPOSE

Key terms: scene = saved node tree; node = object in that tree; signal = event notification; HUD = screen-space information; phase = named match state; CPU intent = action values produced by controller.

HOW THE FLOW WORKS

The diagrams describe the current local project architecture. They are intended for presentation and study, not as a substitute for reading the code when changing behavior.

PRESENTER NOTE

Close with the system loop: input, simulate, resolve, present, then continue the round or match.

PROJECT FILES project.godot | scenes/game/main.tscn | scenes/arenas/arena.tscn | scenes/player/fighter.tscn | scripts/game/main.gd | scripts/player/fighter.gd